DuraShare: BIP39-Native Threshold Backup over GF(2053) with Full Manual Fallback and Per-Share Audit

Renato Schiavinato Lopez
GRIFORTIS
renato@grifortis.com

July 2026
v0.7.0*

Abstract

Long-horizon self-custody needs backups that can survive lost devices, obsolete software, and unavailable vendors. DuraShare is a BIP39-native k-of-n threshold backup scheme that applies standard Shamir sharing directly to BIP39 word indices over GF(2053), the smallest prime field containing the 2048-word list. The normal workflow is software-assisted on an offline device, but the stored artifacts remain recoverable from paper, pre-computed Lagrange coefficients, and ordinary modular arithmetic.

Each Share includes public consistency checks forming a distance-4 product-code layer: they detect every passive error pattern affecting up to three cells and give a unique single-cell repair candidate under an explicit single-error assumption. These checks catch arithmetic, transcription, damage, and share-number mistakes; they are not authentication against deliberate replacement.

To address substitution, the protocol supports per-share audit before recovery. Public checks detect passive corruption, optional MAT columns provide bounded manual substitution detection for each word row, and Manifest Audit Hashes provide computational commitment checks when a separate uncompromised Manifest is available. MAT tags and keys remain human-readable artifact fields and do not alter the arithmetic share or digital payload.

Computational profiles add QR transport, a Full-profile Transport Hash, protocol-input-bound recovery tags, and wallet-context checks, while remaining outside the unconditional secrecy claim of the arithmetic layer. The paper specifies the arithmetic construction, audit boundaries, security claims, manual recovery path, output profiles, and reference test vectors.

1 Introduction

Crypto-asset self-custody often depends on a single secret—a BIP39 mnemonic [4]—serving as the master seed for hierarchical deterministic wallets [3]. Usability studies in Bitcoin key management illustrate a broader self-custody problem [11]: if this secret is lost, assets become unrecoverable; if exposed, they can be stolen. Threshold backup schemes address that single-point-of-failure problem, but long-horizon recovery introduces a second requirement: the backup format should remain recoverable even if the original software stack, device vendor, or preferred encoding disappears.

*Specification, test vectors, and in-progress prototype implementations: https://github.com/GRIFORTIS/durashare. Implementations are experimental, unaudited, and may lag the current specification. Responsible disclosure: security@grifortis.com.

1.1 Problem Setting and Deployment Thesis

This work targets *cold storage and disaster recovery*: splitting an existing BIP39 mnemonic into threshold shares stored as durable artifacts, not day-to-day key management or transaction authorization. The recovery horizon may span years or decades, during which the original software, hardware, and custodians may all become unavailable. BIP39 nativeness therefore matters: a recovered mnemonic should work directly in any BIP39-compatible wallet without format conversion or vendor-specific tooling.

The design goal is algorithmic transparency. The protocol shares the BIP39 word index directly over a prime field chosen to contain the full wordlist. There is no key derivation function (KDF), compression step, or proprietary share alphabet between the paper arithmetic and the recovered mnemonic: the manually recovered values are the BIP39 indices themselves.

The recommended deployment is human-first and software-assisted—an offline device generates shares, validates intermediate state, guides artifact production, and later assists recovery so that ordinary users do not need to perform Shamir arithmetic during normal operation. The protocol does not equate computation with permanent software dependence. We use *sovereignty* to mean that recovery does not require the original application, vendor, device, or proprietary encoding. The software-assisted deployment is the practical default for both sharing and recovery; the arithmetic layer specifies manual fallback procedures, and in the extreme a fully manual share-generation ceremony, as the continuity backstop when computation is unavailable or untrusted. The operational asymmetry is deliberate: share creation is a planned ceremony, while recovery may be an emergency; in author-recorded zero-error runs, fully manual recovery took 29–62 minutes. The protocol is designed for both self-directed users and consultant-assisted ceremonies.

1.2 Claim Scope and Non-Goals

DuraShare separates confidentiality, passive fault detection, and active substitution detection. The arithmetic layer is standard Shamir sharing over GF(2053) applied independently to BIP39 word indices. Its confidentiality claim is information-theoretic: any coalition holding fewer than k arithmetic shares learns no information about the protected mnemonic beyond public parameters.

The row checksums, column checksums, and Global Integrity Check (GIC) form a linear consistency layer for passive faults: arithmetic mistakes, transcription errors, damaged cells, and accidental share-number mixups. This layer is not an authentication mechanism. An adversary who can rewrite a share can also recompute consistent linear checks. Active substitution is therefore handled through separate detection and custody layers: optional MAT for manual pre-recovery authentication, manifest audit hashes when an uncompromised manifest is available, protocol-input-bound warning checks after recovery, wallet-bound Recovery Verification Address (RVA) checks, artifact separation, and physical custody procedures. The protocol can flag inconsistent artifacts or wrong recovery results under stated assumptions; it cannot prevent replacement or destruction of physical or digital artifacts.

BIP39 language and wallet identity. BIP39 derives the seed from the mnemonic sentence, not from the index vector alone. The same sequence of word indices rendered through a different BIP39 wordlist language can therefore produce a valid-looking mnemonic that derives a different BIP32 wallet. DuraShare treats wordlist language as wallet-defining recovery context: in BIP39 mode, it is encoded in the Language/Input byte and included in the RBT canonical material. For manual fallback, the selected BIP39 wordlist language is also printed prominently on Share and Manifest headers. This prevents index-based "translation" across BIP39 languages from being treated as the same recovered protocol object. Pre-Encrypted Numeric Input is separate: the protocol shares externally encrypted field elements and outputs encrypted numeric symbols until the user decrypts them outside DuraShare.

The protocol does not protect against compromise of k or more shares, compromised devices or peripherals that observe secret-bearing material, physical coercion, social engineering, loss or corruption of all recovery context, or errors in the underlying BIP39/wallet ecosystem. The fully manual fallback preserves implementation-independent recovery. Without MAT, its pre-recovery checks remain public consistency checks; without RVA, it also lacks a wallet-bound post-recovery witness.

Terminology. We use *arithmetic layer* for the GF(2053) Shamir construction and its row, column, and GIC checks. We use *linear consistency layer* for those row, column, and GIC checks when discussing passive-fault detection. The optional *Manual Authentication Layer* places one or two Manual Authentication Tag (MAT) columns beside the word rows; this choice is independent of the Full/Compact Output Profile. We use *ceremony* for a controlled Sharing, Share Audit, or Recovery operation with named roles, a trusted boundary, ordered checks, STOP conditions, and cleanup. A *Zero-Knowledge Facilitator* may hold confidential operational metadata and one complementary MAT key half, but does not see a mnemonic, BIP39 passphrase, Shamir share values, or complete MAT keys. We use *computational deployment* for the recommended offline-device profile that adds digital verification mechanisms. We use *Output Profile* for the user-facing choice between Full and Compact artifacts. In the byte-level reference this same choice is the *Computational Payload Output Profile*, because it determines digital payload and verification metadata. We use *Language/Input mode* for the user-selectable flag encoded in computational payloads. The same byte carries two distinct choices: whether the protected material is BIP39 or Pre-Encrypted Numeric Input, and, in the BIP39 case, which wordlist language defines the indices. Pre-Encrypted Numeric Input is an advanced mode for externally encrypted field-element payloads; recovery outputs opaque encrypted symbols rather than wallet-ready words. This byte is incorporated into the canonical material bound by the Recovery Binding Tag. We use *digital envelope* for the self-identifying Payload encoding around each share. We use *manual fallback* for recovery, and optionally sharing, from durable artifacts and modular arithmetic without relying on the original software. A *resume artifact* is temporary sharing-continuity material only; it is not recovery material.

The protocol also separates computation trust from artifact-production trust. Four artifact-production tiers (Section 3, Appendix D) define how much paper output may receive secret-bearing material on each output device. In every tier, any peripheral that handles secret-bearing material is part of the trusted environment for that workflow (Section 3.3).

1.3 Contributions

DuraShare makes the following contributions:

1. **BIP39-native threshold-backup format:** a Shamir instantiation over GF(2053)—the smallest prime greater than 2048—that operates directly on 1-indexed BIP39 word indices, preserving standard BIP39 input/output without custom share alphabets or opaque seed transforms.
2. **Human-first computational deployment with full manual fallback:** the normal sharing and recovery workflows use an offline tool to reduce arithmetic and transcription burden, while the same arithmetic layer can be recovered with modular arithmetic and pre-computed Lagrange coefficients, and can be generated fully manually when computation cannot be trusted.
3. **Linear consistency layer:** a q-ary SPC product-code checksum architecture with provable minimum distance $d = 4$, enabling incremental detection of passive arithmetic and transcription errors, and exact single-cell correction under a single-error assumption, during manual fallback workflows.

4. **Manual authentication layer:** an optional one- or two-column linear universal-hash authenticator over GF(2053), verifiable with the same calculator used by manual recovery and supported by Whole-Key or Split-Key Manifest custody.
5. **Layered deployment model:** a computational deployment profile that adds Output Profiles, a transport hash in the Full profile, protocol-input-bound warning tags, wallet-bound post-recovery checks, and explicit artifact separation without folding those metadata into the information-theoretic secrecy claim.
6. **Metadata and LLR positioning:** placement of manifest audit metadata, DRK coefficient derivation, recursive composition, and small-prime-field concerns within the local-leakage-resilience framework for small-field secret sharing.
7. **Security and reproducibility analysis:** a treatment of arithmetic-share confidentiality, substitution-detection limits, the post-quantum status of the protocol's computational components, advanced recursive custody topologies, and version-scoped reference artifacts for independent replication.

1.4 Paper Organization

Section 2 reviews the mathematical background and related work. Section 3 defines the deployment model, artifact classes, and threat model. Sections 4–6 specify the arithmetic construction, linear consistency layer, and Manual Authentication Layer; Section 7 specifies the computational deployment. Section 8 analyzes confidentiality, metadata and resume mechanisms, and substitution detection. Sections 9–11 cover reproducibility, limitations, future work, and conclusions. Appendix A lists pre-computed Lagrange coefficients, Appendix B gives a worked example, Appendix C summarizes the digital envelope profiles, and Appendix D summarizes deployment and artifact trust boundaries.

2 Background and Related Work

2.1 Shamir's Secret Sharing

Shamir's Secret Sharing (SSS) [22] splits a secret s into n shares such that any k suffice for reconstruction and any $t < k$ reveal zero information about s.

Definition 2.1.1 (Shamir Sharing over GF(p))**.** *Given secret $s \in \mathrm{GF}(p)$, threshold k, and n share indices $x_1, \ldots, x_n \in \mathrm{GF}(p) \setminus \{0\}$:*

1. *Sample $a_1, \ldots, a_{k-1} \xleftarrow{\$} \mathrm{GF}(p)$ independently and uniformly.*
2. *Define $f(x) = s + a_1 x + \cdots + a_{k-1} x^{k-1} \pmod p$.*
3. *Output shares $(x_i, f(x_i))$ for $i = 1, \ldots, n$.*

Recovery uses Lagrange interpolation: $s = f(0) = \sum_{j=1}^{k} \gamma_j \cdot y_j \pmod p$ where $\gamma_j = \prod_{i \neq j} \frac{x_i}{x_i - x_j} \pmod p$.

Security. Information-theoretic (unconditional) secrecy: for any two candidate secrets s_0, s_1, the distribution of any $t < k$ shares is identical. This holds regardless of adversary computational resources.

2.2 Linear Secret Sharing Schemes

A Linear Secret Sharing Scheme (LSSS) [1] is one where share values are linear functions of the secret and dealer randomness. In the span-program view, an authorized set can reconstruct exactly when its rows span the target vector; an unauthorized set learns nothing when its rows do not. A participant's share may contain several field elements. Adding deterministic affine functions of field elements already held by the same participant does not enlarge the unauthorized span or add information; it is local post-processing of the same view. This is the sense in which the checksum fields below are treated as part of the arithmetic share. Public or separately aggregated linear metadata is different and is analyzed explicitly in Section 8.3.

2.3 Product Codes and Two-Dimensional Parity

Product codes, introduced by Elias [12], construct a larger block code by arranging symbols in a rectangular array and requiring every row and every column to be a codeword of smaller component codes. If the component codes have minimum distances d_1 and d_2, the product code has minimum distance $d_1 d_2$. The graphical view of such constraints as a bipartite relation between symbol nodes and check nodes is a Tanner-graph representation [27]; in matrix form, it is simply a parity-check matrix for a linear code.

The simplest component code is a single-parity-check (SPC) code, an $[m, m-1, 2]$ linear code over the relevant field. A two-dimensional SPC product code is the familiar row-and-column parity construction: data symbols occupy a rectangle, parity symbols complete each row and column, and the final corner is the parity of the parity symbols. DuraShare uses a q-ary version of this construction over $q = 2053$, with public affine offsets for row labels, column labels, and share-index binding.

2.4 BIP39 Mnemonics

A BIP39 mnemonic [4] encodes cryptographic entropy as a sequence of $\ell \in \{12, 15, 18, 21, 24\}$ words from a standard 2048-word list. Each word maps to an 11-bit index. For 24 words, the mnemonic encodes 256 bits of entropy plus an 8-bit SHA-256 checksum, yielding 2^{256} valid mnemonics out of $2048^{24} \approx 2^{264}$ possible sequences. An optional passphrase ("25th word") participates in seed derivation but is external to the mnemonic encoding.

2.5 Related Work

Multi-Party Computation (MPC) wallets, multisignature schemes, and smart contract social recovery [8] address *operational security* for active asset management. DuraShare addresses a different problem: *cold storage and disaster recovery*—how to keep threshold recovery available across long time horizons without permanently binding that recovery to one software stack. The comparisons below are therefore limited to threshold backup schemes for stored secrets.

Existing threshold backup systems vary along several axes relevant here: whether the recovered secret is native BIP39, whether the original BIP39 wordlist language is preserved as wallet context, whether the share representation admits a specified non-software recovery path, which error-detection mechanisms are available during recovery, and whether the scheme supports per-share pre-recovery verification and post-recovery protocol-input or wallet-bound verification.

Manual-capable schemes. Codex32 [9] (BIP-93) uses BCH error-correction codes and paper computation wheels (volvelles), providing strong manual error handling but requiring non-BIP39 encoding and a more specialized paper procedure. SeedXOR [21] offers a simpler mental model and standard mnemonic output, but remains inherently n-of-n and provides weak error detection. Table 1 compares these schemes with DuraShare on the dimensions most relevant to long-horizon paper recovery.

Table 1: Selected manual-capable mnemonic-backup schemes

Feature	**DuraShare**	**Codex32**	**SeedXOR**
Primary scope	BIP39	Bitcoin/BIP32 only	BIP39
Threshold flexibility	Full k-of-n	k-of-n ($k \leq 9$, $n \leq 31$)	n-of-n only
Manual procedure	Integer arithmetic mod 2053	BCH + volvelles/tables	XOR
Error handling	Detects 3; corrects 1†	Correction (BCH)	Weak/None
Per-share pre-recovery check	Yes†	Error detection only	No
Post-recovery verification	BIP39 + RVA‡	N/A (non-BIP39)	BIP39 only
BIP39 round-trip	Native I/O	No	Yes
BIP39 language context	Explicit; RBT-bound	N/A	External
Share representation	Paper table + optional QR	Custom bech32 strings	BIP39 words
Recursive composition	Yes	Flat	Flat

† Distance-4 q-ary SPC product code (Section 5); optional MAT adds substitution detection, false-accept $\leq 1/2053$ per column (Section 6).
‡ RVA: truncated wallet address compared after recovery, using offline wallet software only (Section 7.2).

Software-oriented schemes. SLIP39 [24] and SSKR [25] are designed for computational recovery. Both are cryptographically sound, but the underlying arithmetic and share encodings are not specified for paper reconstruction by ordinary prime-field arithmetic. SLIP39 additionally uses a key derivation function that is not BIP39-compatible, while SSKR preserves BIP39 round-trip compatibility by operating on raw entropy and encoding shares with Bytewords/UR. Table 2 summarizes the resulting design trade-offs.

Table 2: Selected software-oriented threshold schemes

Feature	**DuraShare**	**SLIP39**	**SSKR**
Arithmetic domain	Prime GF(2053)*	GF(2^8) + RS1024 checksum	Extension GF(2^8)
Wallet compatibility	Native BIP39	Different KDF	Full round-trip
BIP39 language context	Explicit; RBT-bound	N/A	External
Share encoding	Table + optional QR	Custom 20-word	Bytewords/UR
Per-share pre-recovery check	Yes†	Share checksum only	Share checksum only
Post-recovery verification	BIP39 + RBT + RVA‡	Reconstruction digest	BIP39 checksum
Non-software recovery path	Yes (specified)	No	No
Manual arithmetic	Integer mod 2053	No	No

∗ Structurally immune to the Guruswami–Wootters [15] local-leakage attack on Shamir over characteristic-2 extension fields; informational positioning only, since side-channel leakage is outside this paper's threat model (Section 8.3).
† Independent pre-recovery substitution checks via MAT and/or Manifest Audit Hash (Section 6).
‡ RBT verifies the recovered protocol object ($\sim 2^{96}$ Full/$\sim 2^{48}$ Compact); RVA verifies final wallet context ($\sim 2^{60}$) (Sections 7.2, 7.2).

Information-theoretic authentication. MAT instantiates the Wegman–Carter pattern [29]: a universal hash selected by secret random weights and masked by a fresh one-time field pad. For a changed three-element row, verification reduces to one nontrivial linear equation in the hidden weight vector, giving a worst-case false-accept bound of $1/|\,\mathrm{GF}(2053)|$ per independent tag column.

Small-field secret sharing under partial leakage. A separate line of work studies what happens when Shamir's secrecy assumption is relaxed: instead of an adversary holding fewer than k full shares, the adversary observes additional bounded side-channel leakage from the remaining shares. Guruswami and Wootters [15] showed that Reed-Solomon reconstruction techniques translate into local-leakage attacks on Shamir secret sharing over characteristic-2 fields, making it insecure under even one bit of leakage per share. Benhamouda, Degwekar, Ishai,

and Rabin [2] initiated the corresponding positive theory for prime-order fields; subsequent work by Maji, Nguyen, Paskin-Cherniavsky, Suad, and Wang [17], by Klein and Komargodski [16], and by Maji et al. [18] progressively tightened the leakage-resilience thresholds and identified attack structures that depend on the reconstruction threshold k and on the residue $|F| \bmod k$. These results inform the structured-leakage discussion in Section 8.3, even though DuraShare's threat model (Section 3.3) excludes the physical-bit side-channel scenarios these works target. The prime-field choice GF(2053) is structurally on the favorable side of the characteristic-2 versus prime-field dichotomy in this literature.

3 System Model and Threat Model

3.1 Deployment Model

DuraShare has one arithmetic layer and several operational profiles. Full is the recommended Output Profile for long-term, high-value custody: an offline tool generates the shares, encodes the digital envelope with a transport hash and a longer protocol-input-bound tag, and renders the artifacts intended for storage. Compact uses a smaller QR payload for constrained-output ceremonies where QR size, small displays, or hand transcription materially dominate operational risk. Manual recovery, and at the extreme, full manual sharing, are specified as fallback procedures for situations in which software is unavailable, incompatible, or untrusted.

Full and Compact are Output Profiles; both can produce either printed or hand-transcribed artifacts. Compact's smaller QR grid (V3, 29×29 versus V5–6, 37×37–41×41) is motivated not only by payload size but by constrained-output environments—such as small-display air-gapped devices or ceremonies where manual QR transcription is required—in which a smaller grid materially reduces hand-mark burden (Section 7.1).

MAT is orthogonal to the Output Profile. No MAT, single MAT, and dual MAT use the same Compact or Full payload bytes; MAT tags and keys remain human-readable Share and Manifest fields. Software-assisted ceremonies SHOULD use dual MAT when MAT is selected, while fully manual ceremonies may select either column count according to artifact and audit burden.

Orthogonal to the output profile, the protocol defines resume modes for ceremony continuity and four artifact-production tiers ordered by output-device trust: Tier 1 (offline non-network printer, the only tier permitted to receive secret-bearing material), Tier 2 (personal network-capable printer, non-secret Share and Manifest metadata, partially completed forms, and blank QR grids), Tier 3 (untrusted printer, blank structure only), and Tier 4 (no printer, hand-transcription only). Full tier definitions, the artifact-class table, and the lifecycle rules for each class are given in Appendix D.

Independently of the artifact-production path, when no trusted computation device is available, the protocol falls back to fully manual arithmetic: modular arithmetic, pre-computed Lagrange coefficients, and manual transcription. This is the sovereignty-preserving continuity plan, not the recommended default. Computational deployment is operationally superior whenever a trustworthy offline tool is available, because it adds transport hashing, protocol-input binding, and faster operator workflows.

3.2 Artifact Classes and Metadata Scope

The security claims in DuraShare apply to different data with different scopes. These classes do not create additional document types: MAT fields remain on the Share and Manifest. We distinguish:

1. **Arithmetic share layer:** the GF(2053) word-share values together with the derived row checksums, column checksums, and share-bound GIC. This is the object covered by Proposition 8.1.1.

2. **Share-local recovery metadata:** human-readable hints such as label/date, derivation hint, passphrase-required indicator, and RVA. These items do not participate in the sharing arithmetic. Their role is operational and post-recovery verification. Because computational formats encode at most a coarse wallet class, deployments that rely on RVA-based verification require enough wallet-context information to be recoverable from any valid threshold set rather than only from a separate non-threshold record. In the recommended computational deployment, this means replicating the RVA together with a minimal wallet-context hint on each share artifact, while treating any manifest copy as secondary.
3. **Manifest metadata:** session- or set-level metadata such as Session Batch ID, RBT, per-share audit hashes, custodian or storage notes, wallet-context hints, and copied session identifiers. A manifest can improve logistics and pre-recovery checking, but it remains a distinct operational artifact and is assumed to be stored separately from all shares.
4. **MAT material:** human-readable row tags on the Share and their complete or additively split keys on the Manifest. MAT tags are not arithmetic-share coordinates, and MAT keys are secret material for output and handling rules.

This distinction matters because a Manifest contains no plaintext Share values but may contain secret MAT keys. The protocol does not place printed GIC values or GIC maps in the Manifest, so aggregation does not expose an unbound GIC relation. Computational deployments use Audit Hashes for pre-recovery checking; MAT uses independent Manifest randomness and does not alter that rule.

3.3 Threat Model

All cryptographic claims in the sequel are conditional on the following operational model.

Setup models. *Self-directed:* user generates own shares; dealer and user are identical. *Assisted:* a Zero-Knowledge Facilitator provides procedural and continuity support while the user maintains exclusive control of the mnemonic, Share values, and complete MAT keys. The facilitator may hold confidential operational metadata and one complementary Split-Key Manifest half, but MUST NOT hold a Whole-Key Manifest or both complementary MAT halves.

Assumptions. Both models assume: (1) every device that handles secret-bearing material during Sharing, Share Audit, or Recovery—including the primary computation device and any printer, scanner, camera, or electronic calculator used in the workflow—is uncompromised and does not persist or exfiltrate sensitive data; (2) cryptographically secure random number generation; (3) accurate procedure execution; (4) shares are stored securely; and (5) if a manifest is used, it is stored separately from the Shares. MAT additionally assumes that no adversary able to modify a Share obtains the complete Share Weight vector for any MAT column required to pass. Assumption (1) applies only to peripherals that handle secret-bearing material; peripherals used exclusively for non-secret fields or blank templates are not subject to this constraint (Section 7.3). A private, surveillance-resistant environment is desirable in all ceremonies, but especially in manual workflows because of the longer exposure window and physical working artifacts.

Adversary. Standard Shamir adversary obtaining fewer than k shares. Mode-specific adversaries target side-channels, malicious dependencies or firmware, and supply-chain compromise of devices or peripherals that process secret-bearing data (computational deployment), or physical observation, impression attacks, and waste recovery (manual fallback). The MAT model additionally permits a malicious Share custodian or one Split-Key half-holder who later gains write access to a Share; it excludes exposure of the complete Share Weight vector for a required MAT column.

Out of scope. Compromise of k or more shares, physical coercion, compromised devices or peripherals that observe, persist, or exfiltrate secret-bearing data, social engineering, standard Shamir attacks [22, 26], partial-share or physical-bit side-channel leakage from honest custodians (the local-leakage-resilience setting discussed for context in Section 8.3), and underlying BIP39 ecosystem vulnerabilities.

Operational mitigations. The intended deployment model favors air-gapped or minimal-purpose devices, non-networked peripherals for secret-bearing output, offline verification, explicit memory cleanup, and workflows that minimize persistent secret-bearing artifacts. Non-secret Manifest fields follow the existing metadata rules; complete and split MAT key fields follow the secret-material output rules.

3.4 Human-First Ceremonies

The protocol defines three ceremonies: Sharing creates and validates a backup; Share Audit validates one stored Share without combining shares; Recovery reconstructs the protected material from an authorized threshold set. Their purpose is to reduce human error, establish explicit STOP conditions, and keep secret-bearing work inside a controlled boundary while preserving manual fallback.

During Sharing, the workflow validates non-secret setup parameters, creates or validates any temporary resume material before mnemonic entry, validates the mnemonic before polynomial evaluation, computes Shares and consistency checks, generates MAT when selected, verifies transcription, and removes temporary secret-bearing artifacts after completion. During Recovery, the workflow guides Share entry or scanning, validates each Share, selects a threshold set, interpolates the mnemonic, verifies row/column/GIC consistency, validates the BIP39 checksum, and performs RBT/RVA checks when available.

Share Audit Ceremony. A Share Audit handles exactly one physical Share at a time and does not transmit plaintext Share values over a network. The owner or authorized equivalent confirms the artifact and captures one fixed reading, reconstructs split MAT keys privately when needed, validates row/column/GIC checks, verifies every expected MAT column, and then performs Transport Hash and Manifest Audit Hash checks when a digital payload exists. Full payload values are compared against the complete four-column paper table; Compact payload words are compared against every paper word before the paper checks are recomputed. A failed artifact is never edited, retagged, or resubmitted with different values. An unresolved missing, unreadable, corrupted, substituted, or inconsistent Share triggers Recovery followed by a new Sharing Ceremony.

Zero-Knowledge facilitation. A facilitator may coordinate people, custody contacts, wallet context, and succession steps, and may hold a complete operational copy of Split-Key Manifest B. During secret-handling phases, the facilitator remains outside the secret-handling boundary. The owner MAY and SHOULD retain copies of both Manifest A and Manifest B, stored separately where practical, to preserve independent MAT auditing. Giving the only available B copy to a facilitator creates a voluntary dependency for Split-Key MAT audits, never for recovery.

Certain failures MUST stop the ceremony rather than become warnings: failed authentication, malformed resume data, invalid mnemonic input, Transport Hash mismatch, row/column/GIC mismatch, missing or mismatched expected MAT, and runtime consistency assertion failure. Missing optional metadata such as RVA or derivation hints should be surfaced because it reduces future recovery assurance.

Resume artifacts. Resume artifacts are temporary sharing-continuity material only. They exist to continue an interrupted sharing ceremony; they are not recovery inputs and should not

be required after shares have been produced. In the software-assisted sharing workflow, any selected resume artifact is created or authenticated before mnemonic entry, so resume continuity never requires storing the mnemonic itself. Recovery reconstructs the mnemonic from k shares by interpolation and does not consume the original resume artifact or require the original ceremony device.

Digital Resume, Hand-Transcribed Table, and Hand-Transcribed QR are sensitive while they exist. Decrypted coefficient material, or a decrypted Deterministic Resume Key (DRK) combined with corresponding share data, can reconstruct protected word values. After successful share creation and validation, application-managed resume material should be deleted, and externally held resume artifacts should be destroyed or separately secured. No Resume is appropriate when the sharing ceremony is expected to complete in a single session.

Hand-Transcribed QR is the only resume mode that adds a computational assumption to coefficient generation: deterministic derivation from a 256-bit DRK relies on HMAC-SHA256 [19] pseudorandomness. Implementations domain-separate this derivation and bind it to public ceremony context such as protocol version, output profile, layer path, scheme, element position, and coefficient index; the secret value is not part of the HMAC message. This assumption is outside the information-theoretic secrecy claim and applies only to that resume mode. Digital Resume, Hand-Transcribed Table, and No Resume source coefficient material from true random generation.

4 Core Protocol over GF(2053)

All operational profiles share the arithmetic layer specified in this section. The output is a durable share layout whose semantics do not depend on any QR encoding or digital envelope. Later sections analyze the linear consistency properties of this layout and describe the digital envelope layered on top of it in computational deployments.

4.1 Field Selection: GF(2053)

The protocol uses GF(2053), the smallest prime field containing the 2048 BIP39 word indices. We use 1-based indexing (abandon = 1, zoo = 2048), so the protected value for word position i is the word index w_i itself. There is no hash, encrypted blob, binary seed fragment, compression step, or implementation-specific encoding between the sharing arithmetic and the recovered BIP39 mnemonic.

This choice supports implementation-independent recovery: once the field elements are reconstructed, the BIP39 word indices have been reconstructed directly. It also keeps manual fallback arithmetic in ordinary prime-field modular arithmetic, executable with a basic calculator or precomputed tables.

Because $2053 > 2048$, five field elements fall outside the BIP39 word range:

$$\{0, 2049, 2050, 2051, 2052\}.$$

These values may appear as share values, although never as recovered BIP39 word indices in a valid recovery. On share artifacts, in-range values may be displayed alongside their BIP39 words for readability; out-of-range values are represented numerically and should be visually labeled as numeric GF(2053) field elements, not as missing BIP39 words. This does not affect security or recovery correctness.

4.2 Sharing Algorithm

Parameters and notation.

- **Threshold:** (k, n) with $2 \leq k \leq n \leq 2052$ (see Section 4.4 for $k = 1$); share indices must be distinct nonzero elements of GF(2053)

- **Mnemonic:** $(w_1, \ldots, w_\ell) \in \{1, \ldots, 2048\}^\ell$ with $\ell \in \{12, 15, 18, 21, 24\}$
- **Row count:** $r = \ell/3$ (the mnemonic is arranged as r rows $\times$ 3 columns)
- **Share indices:** $x \in \{1, \ldots, n\} \subset \mathrm{GF}(2053) \setminus \{0\}$
- **Row tags:** $\tau_j^R = j$ for $j = 1, \ldots, r$
- **Column tags:** $\tau_c^C \in \{100, 200, 300\}$ for $c = 1, 2, 3$
- **Row total:** $T_R = \sum_{j=1}^{r} j = r(r+1)/2$
- **Column total:** $T_C = 100 + 200 + 300 = 600$ (invariant across configurations)

Row and column tags serve as domain separators ensuring positional binding. Column tags are chosen from $\{100, 200, 300\}$ to be disjoint from row indices $\{1, \ldots, r\}$ for all supported mnemonic lengths ($r \leq 8$), with a much wider numeric margin than the row-tag range to make row/column mistakes more evident during manual verification.

Step 1: Word polynomial generation. For each word w_i, $i = 1, \ldots, \ell$:

- Sample coefficients: $a_1, \ldots, a_{k-1} \xleftarrow{\$} \{0, \ldots, 2052\}$ independently and uniformly.
- Define $f_{w_i}(x) = w_i + a_1 x + \cdots + a_{k-1} x^{k-1} \pmod{2053}$.
- Evaluate: $y_{w_i}^{(x)} = f_{w_i}(x)$ for each share index x.

Step 2: Row checksum computation and incremental validation. For each row $j = 1, \ldots, r$:

$$f_{R_j}(x) = \left(f_{w_{3j-2}}(x) + f_{w_{3j-1}}(x) + f_{w_{3j}}(x)\right) + \tau_j^R \pmod{2053} \tag{1}$$

where $\tau_j^R = j$ is added to the constant term only. No additional randomness is required. At any share index x:

$$y_{R_j}^{(x)} = \left(y_{w_{3j-2}}^{(x)} + y_{w_{3j-1}}^{(x)} + y_{w_{3j}}^{(x)} + j\right) \bmod 2053 \tag{2}$$

This row-level structure enables *incremental validation* during share construction: after computing every three word shares and their row checksum, each share's row values can be immediately verified via Equation (2), catching arithmetic or transcription errors before proceeding to the next row.

Step 3: Column checksum computation. For each column $c = 1, 2, 3$:

$$f_{C_c}(x) = \sum_{\substack{i:\text{word } i \\ \text{is in column } c}} f_{w_i}(x) + \tau_c^C \pmod{2053} \tag{3}$$

At any share index x:

$$y_{C_c}^{(x)} = \left(\sum_{i \in \text{col } c} y_{w_i}^{(x)} + \tau_c^C\right) \bmod 2053 \tag{4}$$

Step 4: Global Integrity Check (GIC). Define the base GIC polynomial by summing all word polynomials and adding the row and column totals to the constant term (enabling addition-only validation paths):

$$f_{\mathrm{GIC}}(x) = \sum_{i=1}^{\ell} f_{w_i}(x) + T_R + T_C \pmod{2053} \tag{5}$$

The *share-bound* GIC on each share adds the share number for index binding:

$$\mathrm{GIC}(x) = \left(f_{\mathrm{GIC}}(x) + x\right) \bmod 2053 \tag{6}$$

Three equivalent validation paths hold at any share index x:

$$\text{GIC}(x) \equiv \sum_{i=1}^{\ell} y_{w_i}^{(x)} + T_R + T_C + x \pmod{2053} \tag{7}$$

$$\equiv \sum_{j=1}^{r} y_{R_j}^{(x)} + T_C + x \pmod{2053} \tag{8}$$

$$\equiv \sum_{c=1}^{3} y_{C_c}^{(x)} + T_R + x \pmod{2053} \tag{9}$$

Authentication boundary. The row checksums, column checksums, and GIC provide passive-fault detection, not authenticity. An adversary who can rewrite a Share can recompute consistent linear checks. The optional Manual Authentication Layer in Section 6 adds keyed pre-recovery authentication to the human-readable word rows. Computational deployments additionally provide Manifest Audit Hashes, while RBT and RVA remain post-recovery protocol-input- and wallet-bound checks.

Step 5: Share assembly. Each share at index x contains $\ell + r + 3 + 1$ field elements:

$$S_x = (y_{w_1}^{(x)}, \ldots, y_{w_\ell}^{(x)},\ y_{R_1}^{(x)}, \ldots, y_{R_r}^{(x)},\ y_{C_1}^{(x)}, y_{C_2}^{(x)}, y_{C_3}^{(x)},\ \text{GIC}(x))$$

For a 24-word mnemonic: $24 + 8 + 3 + 1 = 36$ values per share.

Optional MAT columns extend the rendered human-readable table but are not elements of S_x, are not interpolated, and do not change this count. The largest rendered table is dual MAT with a 24-word Share: six columns by nine rows including the footer.

BIP39 passphrase independence. The BIP39 passphrase is external to the sharing arithmetic and is never encoded in share payloads. Users relying on a passphrase must back it up separately.

4.3 Recovery Algorithm

Input. k shares $S_{x_1}, \ldots, S_{x_k}$.

Step 1: Per-share validation. For each share at index x, verify the share-bound GIC using any path from Equations (7)–(9). The $+x$ term binds the GIC to the share number: if x is mislabeled, validation fails. When MAT and its matching Manifest are available, complete the Share Audit procedure before accepting the Share; MAT values authenticate the stored artifact and are not interpolation inputs.

Step 2: Lagrange coefficient computation and sanity check. Compute the interpolation coefficients

$$\gamma_j = \prod_{i \neq j} \frac{x_i}{x_i - x_j} \pmod{2053}, \qquad j = 1, \ldots, k.$$

Then verify:

$$\sum_{j=1}^{k} \gamma_j \cdot x_j \equiv 0 \pmod{2053} \tag{10}$$

This holds because Lagrange interpolation of $g(x) = x$ at $x = 0$ yields $g(0) = 0$. Failure indicates coefficient error.

Lagrange coefficients are determined entirely by share indices and contain zero secret information. They may be pre-computed for common schemes and published (Appendix A), or computed on any device—even untrusted—without leaking secrets.

Step 3: Row-by-row interpolation with incremental validation. For each row $j = 1, \ldots, r$, interpolate the three word values and the row checksum:

$$\hat{s} = \sum_{m=1}^{k} \gamma_m \cdot y_s^{(x_m)} \pmod{2053}$$

for $s \in \{w_{3j-2},\, w_{3j-1},\, w_{3j},\, R_j\}$. Immediately verify:

$$(\hat{w}_{3j-2} + \hat{w}_{3j-1} + \hat{w}_{3j} + j) \bmod 2053 \stackrel{?}{=} \hat{R}_j$$

A mismatch localizes the inconsistency to the current row, allowing re-entry or recalculation before proceeding. When the full syndrome is consistent with exactly one erroneous cell, Theorem 5.2.5 gives a unique correction candidate; this candidate should be treated as an operator-guided repair aid rather than silent auto-correction under unknown multi-error conditions. This incremental structure is the primary error-catching mechanism during manual recovery.

Step 4: Global validation. After all rows pass, interpolate the three column checksums $\hat{C}_1, \hat{C}_2, \hat{C}_3$ and verify each via Equation (4). If share-bound GIC cells are interpolated, the $+x$ terms cancel automatically by Equation (10), yielding the base value $\hat{G} = f_{\text{GIC}}(0)$. Verify this base GIC using the $x = 0$ forms of Equations (7)–(9):

$$\hat{G} \equiv \sum_{i=1}^{\ell} \hat{w}_i + T_R + T_C \equiv \sum_{j=1}^{r} \hat{R}_j + T_C \equiv \sum_{c=1}^{3} \hat{C}_c + T_R \pmod{2053}.$$

The row tags τ_j^R and column tags τ_c^C do *not* cancel, as they are embedded in the constant terms of the checksum polynomials.

Step 5: Output. Convert recovered word indices $(\hat{w}_1, \ldots, \hat{w}_\ell)$ to BIP39 words using the 1-indexed wordlist.

4.4 Advanced Custody Topologies: Recursive Composition

The construction supports recursive composition for advanced custody topologies: a completed share from layer L becomes the protected object of layer $L + 1$. This profile is intended for institution-scale or high-value custody structures where software handles artifact tracking, validation, and recovery guidance. It is not the recommended baseline for ordinary individual backups, because each layer increases ceremony complexity, metadata management, and recovery coordination risk. Each layer remains standard Shamir over GF(2053) with independently computed row checksums, column checksums, and GIC.

Intermediate-layer semantics. At layer $L > 0$, the "word" values being shared are GF(2053) elements from the parent share, not BIP39 word indices. Values $\{0, 2049, 2050, 2051, 2052\}$ are valid inputs. BIP39 validity is checked *only* at the final layer-0 recovery. RBT binding (Section 7.2) applies only at the outermost layer.

Degenerate case: $k = 1$. When $k = 1$, the polynomial is constant ($f(x) = a_0$) and all shares are identical copies of the parent value. This provides replication for access-control grouping (e.g., "any member of this group can represent the group") without threshold protection at that layer.

Depth limits. The sharing arithmetic itself imposes no nesting limit—any share can be recursively shared indefinitely. The practical depth constraint comes from the digital envelope, which allocates a fixed number of bits for per-layer threshold and index metadata (Section 7.1): Full supports up to 4 active layers; Compact supports up to 2. Manual fallback ceremonies without a digital envelope are not subject to this limit.

MAT across layers. Each active recursive layer uses independent MAT keys and a matching per-layer Manifest section. Audits treat one layer and one Share at a time; MAT never crosses or combines layer boundaries.

5 Linear Consistency Layer

The share layout is intentionally redundant. Its row checksums, column checksums, and share-bound GIC form an affine version of a linear code over GF(2053) that supports incremental validation during manual fallback workflows. This section analyzes that redundancy as part of the arithmetic share layer; it is not an authentication mechanism against deliberate substitution.

5.1 Product-Code View

After subtracting the public affine offsets (τ_j^R, τ_c^C, T_R, T_C, and x) and using equivalent signed check equations, the share table is a q-ary two-dimensional SPC product code over $q = 2053$. For an ℓ-word mnemonic with $r = \ell/3$ rows, the normalized table has r data rows and 3 data columns, augmented by one row-check column, three column-check cells, and the GIC corner. In standard product-code notation this is:

$$\mathrm{SPC}[4, 3, 2]_{\mathrm{GF}(2053)} \times \mathrm{SPC}[r{+}1, r, 2]_{\mathrm{GF}(2053)}.$$

Thus the full human-readable share table has parameters

$$[\,4(r{+}1),\ 3r,\ 4\,]_{\mathrm{GF}(2053)}.$$

For a 24-word mnemonic ($r = 8$), this is a $[36, 24, 4]_{\mathrm{GF}(2053)}$ code. The row numbers, column tags, row total, column total, and share-index term $+x$ are public affine shifts that bind position and share index; they do not change rank, distance, or single-error correction capability.

5.2 Error-Detection Properties

The share table for an ℓ-word mnemonic contains $N = \ell + r + 3 + 1 = 4(r + 1)$ cells, where $r = \ell/3$. The $r + 4$ check equations (r row, 3 column, 1 GIC) define a linear code over GF(2053) after normalization.

Definition 5.2.1 (Check equations)**.** *An error vector*

$$\mathbf{e} = (e_{w_1}, \ldots, e_{w_\ell}, e_{R_1}, \ldots, e_{R_r}, e_{C_1}, e_{C_2}, e_{C_3}, e_{GIC})$$

is undetectable *if it satisfies all $r + 4$ check equations:*

$$e_{w_{3j-2}} + e_{w_{3j-1}} + e_{w_{3j}} = e_{R_j} \qquad \text{for } j = 1, \ldots, r \tag{11}$$

$$\sum_{i \in col\ c} e_{w_i} = e_{C_c} \qquad \text{for } c = 1, 2, 3 \tag{12}$$

$$\sum_{i=1}^{\ell} e_{w_i} = e_{GIC} \tag{13}$$

Theorem 5.2.2 (Independence and rank)**.** *The $r + 4$ check equations are linearly independent over* $\mathrm{GF}(2053)^N$*. The check matrix has rank $r + 4$.*

Proof. Each row check equation (11) contains a unique variable e_{R_j} appearing in no other equation ($j = 1, \ldots, r$). Each column check equation (12) contains a unique variable e_{C_c} ($c = 1, 2, 3$). The GIC equation (13) contains a unique variable e_{GIC}. A system of $r+4$ equations where each has at least one variable exclusive to it is linearly independent. □

Parity-check matrix. Let $H \in \text{GF}(2053)^{(r+4)\times N}$ be the parity-check matrix whose rows are indexed by $R_1, \ldots, R_r, C_1, C_2, C_3, G$. Let $\mathbf{u}_{R_j}, \mathbf{u}_{C_c}, \mathbf{u}_G$ denote the corresponding standard basis vectors in $\text{GF}(2053)^{r+4}$. The columns of H are

$$h_{j,c} = \mathbf{u}_{R_j} + \mathbf{u}_{C_c} + \mathbf{u}_G, \qquad h_{R_j} = -\mathbf{u}_{R_j}, \qquad h_{C_c} = -\mathbf{u}_{C_c}, \qquad h_G = -\mathbf{u}_G.$$

Here $h_{j,c}$ denotes the column corresponding to the word cell in row j and column c. By Definition 5.2.1, an error vector is undetectable if and only if it lies in $\ker H$.

Lemma 5.2.3 (Distance and dependent columns)**.** *For a linear code $C = \ker H$, the minimum Hamming weight of a nonzero codeword equals the minimum size of a linearly dependent set of columns of H.*

Proof. If $\mathbf{z} \in C$ is a nonzero codeword with support S, then $H\mathbf{z}^T = 0$ gives a nontrivial linear relation among the columns indexed by S, so those columns are linearly dependent. Conversely, if a set of columns indexed by S is linearly dependent, a nontrivial relation among them produces a nonzero vector $\mathbf{z} \in \ker H$ supported on S. Thus the smallest nonzero codeword weight equals the smallest dependent-column set size. □

Theorem 5.2.4 (Minimum detection distance)**.** *The minimum weight of a nonzero undetectable error vector is exactly $d = 4$. That is, all error patterns affecting 1, 2, or 3 cells are detected with certainty.*

Proof. Equivalently, this follows from the product-code view: both SPC component codes have distance 2, so the product code has distance $2 \cdot 2 = 4$ [12]. The direct parity-check proof below verifies the exact check matrix used here.

By Lemma 5.2.3, it suffices to show that every set of 1, 2, or 3 columns of H is linearly independent, and that some set of 4 columns is linearly dependent.

One column. No column of H is zero, so no one-column dependence exists.

Two columns. If two nonzero columns are linearly dependent, one must be a nonzero scalar multiple of the other, hence they must have the same support. But all columns of H have distinct supports:

$$\text{supp}(h_{j,c}) = \{R_j, C_c, G\}, \qquad \text{supp}(h_{R_j}) = \{R_j\}, \qquad \text{supp}(h_{C_c}) = \{C_c\}, \qquad \text{supp}(h_G) = \{G\}.$$

Therefore every two-column set is linearly independent.

Three columns. Let m be the number of word columns in the set.

1. If $m = 0$, the columns are distinct signed basis vectors and are linearly independent.
2. If $m = 1$, say the set contains $h_{j,c}$, then any nontrivial dependence must cancel the coordinates G, R_j, and C_c. Among check columns, only h_G can cancel G, only h_{R_j} can cancel R_j, and only h_{C_c} can cancel C_c. Thus a dependence involving $h_{j,c}$ requires all three of those check columns in addition to $h_{j,c}$, for a total of 4 columns.
3. If $m = 2$, say the set contains h_{j_1,c_1} and h_{j_2,c_2}, then the G coordinate forces their coefficients to sum to zero, so their contribution is a nonzero scalar multiple of

$$h_{j_1,c_1} - h_{j_2,c_2}.$$

If the two word columns share neither row nor column, this difference has four nonzero row/column coordinates, which one check column cannot cancel. If they share a row, the

difference equals $\mathbf{u}_{C_{c_1}} - \mathbf{u}_{C_{c_2}}$ and requires two column-check columns. If they share a column, it requires two row-check columns. Hence no three-column dependence exists with $m = 2$.

4. If $m = 3$, suppose there is a nontrivial dependence among three word columns. If any coefficient were zero, this would reduce to a one- or two-column dependence, already excluded. So all three coefficients are nonzero. View the three word columns as edges in a bipartite graph whose left vertices are rows and whose right vertices are columns. The row and column coordinates of the dependence imply that, at every incident vertex, the sum of coefficients on incident edges is zero. Therefore no incident vertex can have degree 1, since then the unique incident coefficient would be forced to zero. But every finite acyclic graph has a vertex of degree 1, so the selected three-edge subgraph must contain a cycle. A bipartite cycle has even length, hence at least 4, impossible with only 3 edges. Thus no three-word dependence exists.

Four columns (achievable). Dependent four-column sets exist, for example:

$$\begin{aligned} h_{j,c} + h_{R_j} + h_{C_c} + h_G &= 0, \\ h_{j_1,c_1} - h_{j_1,c_2} - h_{j_2,c_1} + h_{j_2,c_2} &= 0, \\ h_{j,c_1} - h_{j,c_2} + h_{C_{c_1}} - h_{C_{c_2}} &= 0, \\ h_{j_1,c} - h_{j_2,c} + h_{R_{j_1}} - h_{R_{j_2}} &= 0. \end{aligned}$$

These correspond respectively to the point, rectangle, row-pair, and column-pair undetectable error patterns.

Hence the smallest dependent column set has size 4, so the minimum weight of a nonzero undetectable error vector is exactly $d = 4$. □

Theorem 5.2.5 (Single-cell correction under a single-error assumption)**.** *If a received human-readable share table differs from a valid table in exactly one cell, the syndrome uniquely identifies both the erroneous cell and its error magnitude. Therefore the original cell value can be recovered exactly.*

Proof. Let the single-cell error be e_i at table cell i, with nonzero magnitude $\alpha \in \mathrm{GF}(2053)$. The syndrome is

$$\mathbf{s} = H\mathbf{e}^T = \alpha h_i,$$

where h_i is the parity-check column for that cell. By the two-column part of Theorem 5.2.4, no two parity-check columns are nonzero scalar multiples of each other. Therefore the observed syndrome can correspond to at most one cell position i and one scalar α. Once i and α are identified, subtracting α from the received cell restores the unique valid table at Hamming distance one. □

Correction boundary. Theorem 5.2.5 is a diagnostic and repair statement under an explicit single-error assumption. It is not a safe blind auto-correction rule for arbitrary failed validations: some multi-cell errors, including some three-cell errors, can produce a syndrome that is also consistent with a one-cell error. Implementations may display a single-cell correction candidate to help the operator locate and repair a transcription mistake, but SHOULD require re-entry, rescanning, or explicit operator confirmation unless the workflow has independent reason to assume exactly one erroneous cell.

Corollary 5.2.6 (Undetected-error probability)**.** *Under a uniform random error model over* $\mathrm{GF}(2053)^N$*, the probability that a nonzero error is undetectable is* $1/2053^{r+4}$ *for a mnemonic of* ℓ *words* ($r = \ell/3$)*. Concrete values:*

ℓ	r	*Checks (r+4)*	$P_{undetected}$
12	*4*	*8*	$1/2053^8 \approx 3.2 \times 10^{-27}$
15	*5*	*9*	$1/2053^9 \approx 1.5 \times 10^{-30}$
18	*6*	*10*	$1/2053^{10} \approx 7.5 \times 10^{-34}$
21	*7*	*11*	$1/2053^{11} \approx 3.7 \times 10^{-37}$
24	*8*	*12*	$1/2053^{12} \approx 1.8 \times 10^{-40}$

Proof. By Theorem 5.2.2, the null space of the check matrix has dimension $N - (r + 4)$. The fraction of nonzero vectors in the null space is $(2053^{N-(r+4)} - 1)/(2053^N - 1) \approx 1/2053^{r+4}$. □

Practical note. Under realistic (non-uniform) error models, the deterministic detection of all weight-≤ 3 errors (Theorem 5.2.4) is the primary assurance for manual use; single-cell correction (Theorem 5.2.5) is best treated as an operator-guided localization aid. The $1/2053^{r+4}$ bound applies to worst-case uniform random corruption.

Appendix B illustrates the full workflow on a toy instance over GF(11).

6 Manual Authentication Layer

The optional Manual Authentication Layer (MAT) authenticates the three word-share values in each human-readable row before recovery. It is separate from the Shamir arithmetic and SPC product code: MAT values are not interpolated, do not change the arithmetic Share S_x, and do not alter the distance or correction results in Section 5. MAT is also independent of the Compact/Full Output Profile. Its tags and keys are human-readable fields only; they are not serialized in payloads, QR codes, Transport Hashes, Audit Hashes, or text exports.

6.1 Construction

Let $F = \mathrm{GF}(2053)$ and let

$$W_j^{(x)} = (W_{j,1}^{(x)}, W_{j,2}^{(x)}, W_{j,3}^{(x)}) \in F^3$$

be word row j on Share x. A deployment selects no MAT, single MAT ($m = 1$), or dual MAT ($m = 2$). For every Share x and MAT column $c \in \{1, \ldots, m\}$, sample independently

$$U^{(x,c)} = (u_1^{(x,c)}, u_2^{(x,c)}, u_3^{(x,c)}) \xleftarrow{\$} F^3$$

and a fresh Row Pad $b_j^{(x,c)} \xleftarrow{\$} F$ for every word row. The printed tag is

$$\mathrm{MAT}_j^{(x,c)} = u_1^{(x,c)} W_{j,1}^{(x)} + u_2^{(x,c)} W_{j,2}^{(x)} + u_3^{(x,c)} W_{j,3}^{(x)} + b_j^{(x,c)} \pmod{2053}. \tag{14}$$

Every Share and every MAT column uses fresh, independent weights and pads. MAT randomness MUST be independent of Shamir coefficients, resume material, Session Batch ID, and all other protocol randomness. Each active recursive layer uses its own independent MAT key sets.

Calculator execution. Starting from the Row Pad, add one weighted word and reduce after each step:

$$\begin{aligned} T_0 &= b_j, \\ T_1 &= (T_0 + u_1 W_{j,1}) \bmod 2053, \\ T_2 &= (T_1 + u_2 W_{j,2}) \bmod 2053, \\ \mathrm{MAT}_j &= (T_2 + u_3 W_{j,3}) \bmod 2053. \end{aligned}$$

The largest multiply-add intermediate is $2052^2 + 2052 = 4{,}212{,}756$, within an eight-digit calculator.

Rendered layout. The human-readable table has four columns without MAT, five with single MAT, and six with dual MAT. A 24-word dual-MAT Share is therefore a 6×9 table including the footer. MAT applies only to word rows. The footer retains the three column checksums and printed GIC in its first four cells; MAT-column footer cells are marked as not applicable. Mandatory SPC/GIC verification binds the footer to the tagged words, and a separate footer tag does not improve the worst-case forgery bound. The Share header records whether MAT is absent, single, or dual.

Key and tag placement. MAT tags remain on the Share. The matching Share Weights and Row Pads remain on the per-layer Share List in the Manifest. A 12-word Share needs $3 + 4 = 7$ key values per MAT column; a 24-word Share needs $3 + 8 = 11$. Dual MAT doubles the tag and key counts. Software-assisted ceremonies SHOULD use dual MAT when MAT is selected; fully manual ceremonies may select either count.

6.2 Authentication Bound

Theorem 6.2.1 (MAT false-accept bound)**.** *Suppose an adversary can read and replace a Share and its printed MAT tags, while the Share Weight vector for every required MAT column remains independently uniform conditional on the adversary's view. For one fixed artifact presented for verification, any substitution changing at least one tagged word row is accepted with probability at most*

$$2053^{-m},$$

where $m = 1$ for single MAT and $m = 2$ for dual MAT, provided every selected column is independently keyed and required to pass.

Proof. Condition on the adversary's complete view and choose any changed row. Write its nonzero error vector as $e = (e_1, e_2, e_3) \in F^3 \setminus \{0\}$. For MAT column c, the adversary's chosen tag change fixes a value $\delta_c \in F$. Verification requires

$$U^{(x,c)} \cdot e = \delta_c.$$

For every nonzero e and every δ_c, exactly $|F|^2$ of the $|F|^3$ possible weight vectors satisfy this nontrivial linear equation. The probability is therefore $1/|F| = 1/2053$ per column. Independent columns multiply the bound. Additional changed rows add constraints and cannot enlarge the acceptance set. □

No offline key test. For every candidate weight vector and every observed row/tag pair, exactly one Row Pad makes Equation (14) hold. Since each pad is independent and uniform, the visible Share and tags leave all weight vectors equally likely. Rechecking the same fixed reading does not consume this guarantee; an unresolved mismatch terminates the Share Audit Ceremony.

Lemma 6.2.2 (One Split-Key half preserves the bound)**.** *Suppose an adversary knows every MAT value on Manifest A (or symmetrically Manifest B), together with the Share and printed tags, while the complementary half remains unknown. Then the complementary Share Weight vector remains uniform conditional on that view, and Theorem 6.2.1 applies.*

Proof. Write $U = U_A + U_B$ and $b_j = b_{j,A} + b_{j,B}$. After fixing the known half and any candidate U_B, each observed row/tag pair determines exactly one complementary pad value

$$b_{j,B} = \mathrm{MAT}_j - (U_A + U_B) \cdot W_j - b_{j,A}.$$

The complementary pads are independent and uniform, so every candidate U_B is equally consistent with the transcript. □

Interaction with SPC/GIC. A document-replacement adversary can recompute every public checksum, so SPC/GIC does not improve Theorem 6.2.1's worst-case bound. Its role remains deterministic passive-error detection and mandatory footer binding. Conversely, MAT does not authenticate header or wallet-context fields; those are compared against the Manifest. Anyone holding a Share and its complete Share Weight vector can shift tags to match arbitrary word changes, even without learning the Row Pads. A malicious dealer remains outside the MAT guarantee.

6.3 Manifest Custody

Whole-Key Manifest. The complete per-Share MAT key sets are recorded in the existing Manifest. The completed Manifest is secret material, remains separate from all Shares, and follows the same trusted-device and output-tier rules as other secret material.

Split-Key Manifests. For every complete MAT key value $z \in F$, draw a fresh independent $\rho \xleftarrow{\$} F$ and record

$$z_A = \rho, \qquad z_B = z - \rho \pmod{2053}$$

on Manifest A and Manifest B respectively. The trusted owner reconstructs $z = (z_A + z_B) \bmod 2053$ during audit. The two Manifests are complete operational counterparts; only their MAT sections differ. The MAT values in either half are uniform and reveal no complete key.

The owner MAY and SHOULD retain copies of both A and B, stored separately where practical. A Zero-Knowledge Facilitator may retain another complete operational copy of Manifest B for continuity and inheritance, but MUST NOT hold a Whole-Key Manifest or both complementary MAT halves. Giving the only available B copy to a facilitator creates a voluntary dependency for Split-Key MAT audits; Recovery never requires MAT or a Manifest. Even if one half-holder later gains access to a Share, the unknown complementary weights preserve the false-accept bound in Theorem 6.2.1; a half-holder can nevertheless destroy, withhold, or corrupt an artifact and cause denial.

6.4 Share Audit Ceremony

A Share Audit validates exactly one physical Share without combining Shares or recovering the mnemonic. The owner or authorized equivalent controls the secret-handling boundary; plaintext Share values are not transmitted over a network. A facilitator may coordinate the ceremony and provide Manifest B but does not see the Share or reconstructed MAT keys.

The audit order is:

1. confirm the backup, Share number, layer path, word count, and expected MAT selection against the Manifest;
2. capture one fixed reading of the physical Share;
3. reconstruct Split-Key MAT values privately when required;
4. validate every row checksum, column checksum, and the printed GIC;
5. recompute every expected MAT tag, requiring both columns when dual MAT is selected;
6. when a digital payload and Audit QR exist, validate the Transport Hash if present, compare every serialized value with the fixed paper reading, and then compare the payload with the Manifest Audit Hash.

For Full, the digital comparison covers the complete four-column canonical arithmetic table. For Compact, it covers every word value, after which the paper row checksums, column checksums, and printed GIC are recomputed and compared. Repeating arithmetic over the same fixed reading is permitted to exclude operator error; a failed artifact is never edited, retagged, or resubmitted with different values.

An unresolved missing, unreadable, corrupted, substituted, or inconsistent Share triggers Recovery. After validated recovery, the owner either re-shares the existing seed if confidentiality remains credible or moves funds to a new seed if compromise cannot be excluded. The replacement backup is created and independently validated before controlled artifacts from the previous backup are retired and destroyed. Missing or stolen artifacts remain recorded as potentially exposed.

A missing or unreadable Manifest makes its MAT and Audit Hash checks unavailable; it is not by itself evidence that the Share is invalid. If no independent audit path remains, the safe lifecycle is Recovery followed by a new Sharing Ceremony.

7 Computational Deployment and Verification

The arithmetic share layer is format-agnostic, but the recommended deployment wraps each share in a digital envelope for QR transport and stronger verification. Appendix C summarizes the active envelope profiles; byte-level encoding is an implementation reference maintained with the test vectors. The mechanisms in this section are computational checks layered on top of the arithmetic share layer; they are not part of the information-theoretic secrecy claim of Proposition 8.1.1.

MAT remains outside the digital envelope. Its selection, tags, complete keys, and key halves are not serialized by either active Output Profile and do not change the payload sizes below.

7.1 Output Profiles and Manual-Transcription Trade-Offs

Two Output Profiles balance verification strength, payload size, and manual transcribability. In the byte-level specification this choice is the Computational Payload Output Profile. Full is the default for long-term, high-value custody. Compact is an optional profile for constrained-output ceremonies where QR size, small displays, or hand transcription materially dominate operational risk:

Table 3: Computational deployment output profiles

Property	**Full**	**Compact**
Arithmetic share values	Full GF(2053)	Full GF(2053)
QR symbol, 24-word (ISO)	V6 (41 × 41)	V3 (29 × 29)
Hand-mark burden (24-word)‡	∼685 dark modules	∼305 (≈55% fewer)
Pre-recovery transport hash	Yes	No
Protocol-input-bound RBT	$\sim 2^{96}$	$\sim 2^{48}$
Nesting metadata	Up to 4 layers	Up to 2 layers

‡ **Hand-mark burden** counts dark modules in the payload-dependent (data + error-correction) region of *one* share QR after pre-printing ISO structural patterns (three finder squares with separators, timing strips, alignment pattern, format-information strips) on a blank template. Representative 24-word payloads: 99-byte Full (`SF`, V6-M) and 53-byte Compact (`SC`, V3-L); 12- and 15-word Full payloads use V5-M (37×37). See Section 7.1 and Appendix C.1. Quiet zone (four-module margin) excluded. This is a structural workload estimate, not a timed usability result; mask and payload choice may shift counts by ±20–40 modules. Hand-Transcribed DRK resume QRs are separate resume artifacts, not share payloads.

Compact profile: hand-mark workload. Compact reduces ceremony burden when share QRs are hand-transcribed from a trusted display onto a pre-printed structural template (Tier 3/4, Appendix D). Table 3 summarizes the trade-off using *dark modules to mark* in the payload-dependent region—the primary physical transcription cost when light modules remain blank on the template. For the representative 24-word share payloads above, that burden drops from ∼685 modules (Full, 41 × 41) to ∼305 (≈55% fewer). Symbol area alone shrinks by ∼50%

(1,681 → 841 modules), but area reduction overstates transcription savings because finders, timing, and alignment are largely pre-printed on both templates. Compact achieves its smaller QR by reducing digital-envelope metadata: it omits the Transport Hash, serializes word shares only (omitting checksums and GIC from the payload), and uses a shorter RBT. Full remains the default recommendation for long-term, high-value custody whenever the larger QR/transcription burden is acceptable.

Version note. Earlier drafts included a range-restricted Reduced profile that used sample rejection to fit 11-bit word-share values. That design is not part of v0.7.0. The current Compact profile uses unrestricted GF(2053) share values and achieves the V3-L QR budget by reducing digital-envelope metadata. Historical analysis of the deprecated range-restricted profile remains in the repository as design-rationale material; it is not part of the current protocol.

7.2 Verification Layers

Transport Hash (Full). Full includes a truncated SHA-256 [13] hash over the serialized Payload body before the Transport Hash field. This detects random bit flips, scanning errors, or physical damage in the digital envelope before interpolation. It is a computational check on the digital envelope, not part of the arithmetic secrecy claim.

Recovery Binding Tag (RBT). Each computational share payload carries a Session Batch ID and a Recovery Binding Tag (RBT). The RBT is derived by PBKDF2-HMAC-SHA512 [19] with the canonical secret material as password, the Session Batch ID as salt, 16,384 iterations, and a 32-byte output; Full uses the first 12 bytes and Compact uses the first 6 bytes. Canonical secret material is the Language/Input byte followed by the exact byte encoding of the Shamir input symbols at $x = 0$: the 1-indexed BIP39 word-index vector in BIP39 mode, or the ciphertext field-element vector in Pre-Encrypted Numeric mode. After recovery, software recomputes the RBT from the recovered material and compares it to the payload field; a mismatch indicates share mixing, corruption, or substitution of the recovered protocol object. The canonical material is never serialized outside the RBT derivation. For an independently wrong recovered object, accidental matching probability is $\sim 2^{-96}$ in Full and $\sim 2^{-48}$ in Compact.

Wallet-bound verification (RVA). The RBT is bound to the canonical protocol input; it does not by itself verify wallet context, derivation settings, or BIP39 passphrase use. As a complementary check, the protocol allows recording a Recovery Verification Address (RVA): a truncated address derived from the intended target wallet and compared after recovery. For Bitcoin bech32/bech32m addresses [5, 6], the "first 8 characters + last 8 characters" form gives about 2^{60} matching strength under the address-format assumptions analyzed here. Other asset ecosystems require format-specific matching and privacy analysis. If RVA-based verification is relied upon, enough wallet-context information must be recoverable from any valid threshold set; in the recommended computational deployment, the RVA and a minimal wallet-context hint are therefore replicated on each share artifact, while any manifest copy is secondary.

7.3 Manifest-Based Pre-Recovery Audit

An optional Manifest can store session metadata and per-share Audit Hashes without containing plaintext Share values. When stored separately from the Shares, these hashes provide the computational part of the Share Audit Ceremony in Section 6.4: a trusted device scans or imports the payload, validates available local checks, compares every serialized value with the fixed paper reading, recomputes the Payload hash, and compares it with the Manifest record.

The Audit Hash commits to the Share Payload bytes: header fields, session metadata, and serialized Share data. In Full, Share data is the complete canonical arithmetic table (word-share

values, row checksums, column checksums, and printed GIC). In Compact, Share data is word-share values only; row and column checksum cells are not serialized and must be recomputed from the scanned words and compared with the paper. MAT fields are not part of either Payload or Audit Hash input. Manifest-based audit is conditional on artifact separation: an adversary who can alter both a Share and its Manifest record can defeat the computational commitment.

8 Security Analysis

This section separates information-theoretic secrecy of the arithmetic Share layer, MAT authentication under secret Manifest keys, metadata and resume mechanisms outside the arithmetic claim, and substitution-detection properties that depend on operational artifacts or computational checks.

8.1 Confidentiality

Proposition 8.1.1 (Information-theoretic confidentiality of the arithmetic share layer)**.** *For any $t < k$ and any two BIP39-valid mnemonics M_0, M_1, the distribution of any t arithmetic shares is identical, where an arithmetic share consists of the* GF(2053) *word-share values together with the derived checksum fields (row checksums, column checksums, and GIC).*

Proof sketch. The ℓ word indices are protected by independent Shamir polynomials of degree at most $k-1$ with uniformly random coefficients over GF(2053). By Shamir's theorem [22], the distribution of $t < k$ word-share values is identical for any constant term. At each observed share index, the checksum fields are deterministic affine functions of the word-share values at that same index and public offsets (τ_j^R, τ_c^C, T_R, T_C, x). They are therefore redundant local coordinates of the same share, not independent evaluations or public equations on the mnemonic. By LSSS theory [1, 10], restricting the secret domain to BIP39-valid mnemonics does not change this: perfect secrecy is distribution-agnostic. □

Scope. Proposition 8.1.1 applies to the arithmetic Share layer. MAT tags and keys are separate authentication material; RBT and RVA are verification metadata. None is part of the arithmetic Share definition.

Corollary 8.1.2 (Linear consistency fields add no information)**.** *Conditional on the t word-share values held by an unauthorized coalition, the corresponding row checksums, column checksums, and share-bound GIC are deterministic functions of those word shares. In particular, H(checksums | t word shares) $= 0$, so adjoining the checksum fields to the adversary's view preserves the identical-distribution property of Proposition 8.1.1.*

Proof. By Equations (2), (4), and (7), each checksum field at share index x_i is a deterministic linear combination of word-share values at the same index plus public offsets (τ_j^R, τ_c^C, T_R, T_C, x_i). The unauthorized coalition holds exactly the word-share values at its t share indices, so it can recompute every checksum field locally without additional information from the dealer or the unseen shares. Equivalently, although a row-check polynomial has coefficients equal to the sums of the corresponding word-polynomial coefficients, a single share holder observes only its value at one index; without the row secret, that coefficient sum is not isolated by the checksum value. □

Corollary 8.1.3 (MAT non-interference for complete Shares)**.** *Adding MAT tags to each held Share and independent MAT keys to the Manifest preserves Proposition 8.1.1 for any coalition holding $t < k$ complete Shares.*

Proof. Without the keys, each MAT tag is uniform because its fresh Row Pad is uniform. Given the complete matching keys, each tag is a deterministic function of word-share values already

held by the coalition. The complete keys are independent ceremony randomness, and either additive key half is itself uniform. Thus adjoining these values does not distinguish candidate mnemonics beyond the coalition's existing complete-Share view. □

Computational verification. As a reproducibility artifact complementing Proposition 8.1.1, the script `research/security-validation/llr-uniformity/llr_uniformity.py` performs exhaustive enumeration over GF(2053) for several (k, n) configurations: at each configuration it samples one Shamir polynomial, fixes any $k-1$ of the resulting shares as the adversary's view, and verifies that all 2053 candidate secrets are consistent and that the map $s \mapsto f(x^\star)$ is a bijection on GF(2053) at every unseen index $x^\star$. The check is complete (not statistical) over the field, dependency-free, and runs in well under a second. It serves as a computational sanity check on Proposition 8.1.1 over the specific field used by the protocol, including the $k = 5$ case raised informally during external review.

Proposition 8.1.4 (Entropy preservation)**.** *Effective keyspace remains* $\approx 2^{256}$ *for 24-word mnemonics. Sharing over* GF(2053) *introduces no compression.*

Justification. BIP39 entropy for $\ell = 24$ is 256 bits, determining 2^{256} valid mnemonics. Each word index $w_i \in \{1, \ldots, 2048\} \subset \mathrm{GF}(2053)$ is mapped injectively into the field without compression. The ℓ independent Shamir polynomials operate on these indices word by word; no entropy is combined, truncated, or hashed during sharing. The linear consistency fields are deterministic linear functions of the word shares and add no entropy of their own. □

Note on the coupled-constraint question. A previous version of this work [20] posed confidentiality under coupled BIP39/linear constraints as an open question. We observe that this follows directly from LSSS theory: Shamir's perfect secrecy is distribution-agnostic, and the linear consistency fields add no information beyond the t word share values already provided. We invite independent formal verification.

8.2 Substitution Detection

Linear consistency layer: zero substitution detection. Following the cheater framework of Tompa and Woll [28], an adversary with physical access to one share can choose arbitrary word values and compute consistent row checksums, column checksums, and GIC. The modified share passes all $r + 4$ check equations. After Lagrange interpolation with the fake share and $k-1$ genuine shares, the recovered result is internally consistent (linearity is preserved) but produces a wrong mnemonic. The linear consistency layer therefore detects passive faults, not deliberate substitution.

Pre-recovery detection (Manifest-dependent). If the matching Manifest is available, a Share Audit can apply:

1. **MAT** (manual authentication): after mandatory SPC/GIC validation, recompute every expected word-row tag. The false-accept probability is at most $1/2053$ for single MAT or $1/2053^2$ for dual MAT under Theorem 6.2.1.
2. **Audit Hash** (computational deployment): hash the scanned or imported Payload and compare it against the Manifest record. This detects mismatch from the Manifest commitment unless the adversary also compromises the record or finds a hash collision.

These checks have different compromise conditions. An adversary with a Share and its complete Share Weights can forge MAT. An adversary who can alter both a Share Payload and its Audit Hash record can defeat the computational commitment. Manifest content contains no plaintext Share values and no GIC map, but Whole-Key MAT sections are secret material. Session identifiers, RBTs, Audit Hashes, custodian notes, and wallet-context hints remain operationally sensitive and may assist targeting or complete-candidate testing.

Post-recovery detection (manifest-independent). After Lagrange interpolation produces a candidate mnemonic, several checks can flag wrong recoveries without relying on a manifest:

1. **BIP39 checksum** (∼99.6% for 24 words): an independent non-linear check on the recovered mnemonic. It detects many wrong recoveries, but it is not an authentication tag.
2. **RBT check** (computational deployment): because the Session Batch ID and RBT are present in each share payload, the check is available even when the manifest is lost. For an independently wrong recovered object, the RBT matches accidentally with probability about 2^{-96} in Full or 2^{-48} in Compact.
3. **RVA** (all profiles): for the Bitcoin bech32/bech32m truncation analyzed in Section 7.2, an independently wrong recovered wallet context matches with probability about 2^{-60}. Other address formats require their own matching-strength analysis.

These checks require only the recovered mnemonic and recorded share-local metadata. They are post-recovery filters, not proof that the input shares were authentic.

In manual fallback, MAT provides the pre-recovery substitution check when selected. Without MAT, pre-recovery validation remains public arithmetic consistency; without RVA, the recovered wallet context has no wallet-bound witness beyond the BIP39 checksum and operational controls.

8.3 Local Leakage Resilience and Composition

Proposition 8.1.1 establishes information-theoretic confidentiality for any $t < k$ shares of the arithmetic layer in the textbook secret-sharing model. This subsection positions the GF(2053) choice within the local-leakage-resilience (LLR) literature [2, 17, 16, 18] and summarizes the protocol element that sits outside the unrestricted arithmetic layer: DRK-based deterministic coefficient derivation. Scenarios in which the adversary additionally observes partial-share side-channel leakage from honest custodians remain out of scope of this paper's threat model (Section 3.3, assumption (1)).

LLR framework. The local leakage model considers an adversary that holds $t < k$ full shares and additionally observes a bounded leakage function $\mathrm{Leak}_i : \mathrm{GF}(p) \to \{0,1\}^m$ applied independently to each of the remaining $n - t$ honest shares. For Shamir over prime-order GF(p) with random evaluation places, Benhamouda et al. [2] proved leakage resilience for reconstruction-threshold ratios in a high regime; subsequent work by Maji et al. [17, 18] and by Klein and Komargodski [16] progressively reduced the required ratio, with the strongest results favoring high t/n. Over characteristic-2 fields, Reed-Solomon reconstruction techniques due to Guruswami and Wootters [15] render Shamir secret sharing insecure even at $m = 1$ bit of leakage per share. The prime-field choice GF(2053) in DuraShare is structurally on the favorable side of this dichotomy: none of the characteristic-2 attack family applies. As an empirical deployment data point, two widely used BIP39-adjacent threshold schemes operating over even smaller fields—SLIP-39 [24] (in production at SatoshiLabs/Trezor since 2019) and SSKR [25] (Blockchain Commons, since 2020), both over GF(2^8)—have seen substantial production use without a published standard-model break, consistent with the threshold-agnostic, field-size-independent secrecy claim of Proposition 8.1.1.

Structured metadata versus general leakage. The general LLR theorems above bound resilience against *arbitrary* leakage functions. The protocol elements below are not arbitrary physical leakage functions; they are fixed, publicly specified metadata or derivation choices whose consequences can be bounded directly.

- **Manifest metadata and MAT keys.** Manifests do not include printed GIC values or GIC maps, so they do not expose a Manifest-aggregated GIC relation. Session identifiers, RBTs, Audit Hashes, custodian notes, and wallet-context hints remain operationally

sensitive metadata. MAT keys are independent randomness: complete keys enable authentication forgery but reveal no mnemonic information; either Split-Key half is uniform. Corollary 8.1.3 states the complete-Share confidentiality scope.

- **Hand-Transcribed QR (DRK resume).** Coefficients are derived deterministically from a 256-bit Deterministic Resume Key via HMAC-SHA256 (Section 3.4). This is not local leakage from shares; it is structured computational coefficient generation. Under the standard PRF assumption for HMAC-SHA256 [19], the resulting coefficients are computationally indistinguishable from the True Random distribution. This shifts the secrecy claim for this resume mode from information-theoretic to computational, with a distinguishing advantage bounded by the PRF security parameter ($\approx 2^{-256}$ classically, with a post-quantum key-search margin of $\approx 2^{-128}$ via Grover search on the DRK).

Composition across nested layers. DuraShare supports recursive composition (Section 4.4). Each layer is an independent Shamir instance over GF(2053) with independent coefficient randomness and, when selected, independent MAT keys. The digital envelope caps active nesting at 4 layers (Full) or 2 layers (Compact). Metadata risks compose operationally across layers: more Manifest sections, session identifiers, Audit Hashes, and custody notes improve administration but increase targeting and recovery-sabotage surface. Deployments that require maximum confidentiality of metadata should minimize Manifest contents and store Manifests separately from all Shares.

Threshold-modular structure of one physical-bit attack. Maji et al. [17] exhibit a specific physical-bit attack at $m = 1$ bit of leakage per share whose attack structure depends on the residue $|F| \bmod k$ being equal to 1. For GF(2053):

k	2	3	4	5	6	7	8	9	10	11	12
$2053 \bmod k$	1	1	1	**3**	1	2	5	1	3	7	1

The structural condition $|F| \equiv 1 \pmod{k}$ is met at $k \in \{2, 3, 4, 6, 9, 12\}$ and not at $k \in \{5, 7, 8, 10, 11\}$. This is a side-channel attack on unseen shares and falls outside this paper's threat model; it is included for orientation. Two observations follow. First, the specific small-field attack often associated with low-threshold concerns does *not* structurally apply at $k = 5$, the threshold around which earlier informal scepticism was raised. Second, the protocol's prime-field choice is structurally immune to the characteristic-2 LLR attack family [15] regardless of threshold.

Where high thresholds sit, and threshold caps. The proven LLR resilience bounds for prime-field Shamir favor *higher* reconstruction-threshold ratios t/n. The protocol allows extreme thresholds such as 254-of-255 (necessarily $n \leq 2052$, since share indices must be distinct nonzero elements of GF(2053)); such a deployment has $t/n \approx 0.992$, well above the high-ratio regimes covered by the prime-field LLR literature. Conversely, the low-threshold topologies emphasized in this paper (2-of-3, 2-of-4, 3-of-5) sit at $t/n \in \{1/3, 1/4, 2/5\}$, below the proven-resilient regime for arbitrary one-bit physical-bit leakage. This does not contradict Proposition 8.1.1: standard Shamir secrecy is unconditional and threshold-agnostic. It does indicate that, if a deployment's threat model were extended to admit side-channel partial-share leakage from honest custodians, low-threshold topologies would require stronger operational mitigations than high-threshold ones. No additional protocol-level cap is imposed beyond the existing $n \leq 2052$ from share-index distinctness.

8.4 Post-Quantum Considerations

The information-theoretic confidentiality of Proposition 8.1.1 holds against any adversary regardless of computational power, including quantum adversaries: no hardness assumption

is invoked, and no quantum speedup applies to Lagrange interpolation, the linear consistency layer, or the MAT false-accept bound over GF(2053). This subsection enumerates the remaining computational components of the protocol and records their post-quantum status explicitly.

Deterministic Resume Key mode. Hand-Transcribed QR's deterministic coefficient derivation relies on HMAC-SHA256 [19]. Under Grover's algorithm [14], brute search over the 256-bit DRK has effective quantum cost $\sim 2^{128}$ (against $\sim 2^{256}$ classically), and HMAC's PRF property is not weakened beyond that key-search bound. The secrecy claim of this resume mode is therefore approximately 128-bit post-quantum.

Transport Hash, Recovery Binding Tag, and Manifest Audit Hash. The Transport Hash and Full-profile Audit Hash are truncated or full SHA-256 [13]; the RBT is PBKDF2-HMAC-SHA512 [19] with 16,384 iterations. SHA-256 preimage resistance under Grover's algorithm is $\sim 2^{128}$. PBKDF2-HMAC-SHA512 key-search cost under Grover is $\sim 2^{128}$ on the inner SHA-512 state; the iteration count adds a constant factor. The shortest RBT is 6 bytes in Compact, so it should be treated as a warning and mixup-detection mechanism rather than a high-strength authentication tag. Importantly, an adversary who broke any of these primitives—quantumly or classically—still cannot recover the mnemonic from $t < k$ shares, because the secrecy claim of Proposition 8.1.1 is information-theoretic and independent of these checks.

Recovery Verification Address. The RVA inherits the cryptographic properties of the protected wallet ecosystem (e.g., the Bitcoin bech32/bech32m address derivation [5, 6]). It is not an independent cryptographic primitive of this protocol; its post-quantum behavior reduces to that of the underlying wallet chain.

Out of scope. The post-quantum security of the BIP39-derived wallet itself—most notably the ECDSA/EdDSA signing keys whose underlying discrete-logarithm and factoring problems are broken by Shor's algorithm [23] on a sufficiently capable quantum computer—is a downstream property of the wallet ecosystem, not of this protocol. DuraShare does not introduce its own quantum-vulnerable primitives beyond those enumerated above, and it does not solve or alter the quantum status of the secret it protects.

Net status. The arithmetic Share layer with true-random coefficients (Proposition 8.1.1) is unconditionally secure against quantum adversaries, and MAT is information-theoretic under its key-custody assumptions. DRK resume mode replaces true-random coefficient storage with PRF-derived coefficients and therefore relies on HMAC-SHA256 pseudorandomness; Transport Hash, RBT, and Manifest Audit Hash remain operational verification mechanisms, not the basis of arithmetic secrecy. The dominant long-horizon quantum exposure lies in the protected wallet ecosystem rather than in the sharing layer.

9 Evaluation and Reproducibility

This section reports the current reproducibility artifacts and preliminary empirical evidence. Version-scoped test vectors are intended to support independent verification of the protocol. Prototype software exists to support experimentation and implementation work, but it should not be treated as a conformant reference for the current specification unless explicitly versioned as such. The timing and usability observations below are descriptive, author-generated measurements rather than controlled studies.

Prototype implementations. Prototype implementations are in progress in three forms:

- JavaScript/TypeScript library (`@grifortis/schiavinato-sharing`)
- Python library
- Offline single-file HTML tool for air-gapped environments

All implementations are open-source (MIT), experimental, unaudited, and intended for learning, replication, and review rather than operational deployment. Current prototypes cover the core Shamir arithmetic and validation logic, but lag the complete v0.7.0 workflow described here, including MAT, Share Audit, QR payload generation, resume modes, companion-app handling, and printer-tier flows. For that reason, no end-to-end software-assisted ceremony timing is reported in this version.

Test vectors. Version-scoped test vectors in `test_vectors/` provide reproducible fixtures for independent checking, including polynomial coefficients, Share values, and expected checksums. Conformant MAT vectors additionally require Share Weights, Row Pads, expected single/dual tags, and Split-Key reconstruction values.

Author-recorded manual timing observations. Table 4 summarizes zero-error manual runs recorded by the author. These are not controlled usability results and do not describe the recommended software-assisted workflow. They are included only as order-of-magnitude feasibility evidence: sharing can be planned, paused, and software-assisted by default, while fully manual recovery is the critical continuity fallback if the original toolchain is unavailable.

Table 4: Author-recorded manual timing summary

Workflow	**12-word**	**24-word**	**Longest checkpoint**
Fully manual recovery	29 min	62 min	6 / 10 min
Sharing, no MAT	1 h 51 min	4 h 24 min	13 / 20 min
Sharing + single MAT, Whole-Key Manifest	2 h 42 min	6 h 59 min	13 / 20 min
Sharing + dual MAT, Split-Key Manifest	4 h 39 min	11 h 44 min	13 / 20 min[†]

[†] MAT work is naturally checkpointed by Share, MAT column, and word row; in these observations, added MAT subtasks were shorter than the longest share-transcription task.

The observed recovery runs were substantially shorter than fully manual sharing: approximately 30–60 minutes end-to-end, with row-level checkpoints of about 4–5 minutes. These timings support the intended continuity property: emergency recovery remains feasible from durable artifacts and modular arithmetic, while fully manual sharing is more demanding, normally planned, and naturally checkpointed after random generation, each row, each Share, Manifest work, and cleanup. Controlled third-party usability testing remains necessary.

Qualitative QR transcription context. As a qualitative reference point, the author has repeatedly hand-transcribed SeedSigner-style QR codes from a Raspberry Pi Zero display with 240×240 pixels and successfully decoded the resulting QRs. This is not a controlled experiment and does not substitute for DuraShare-specific usability testing, but it informs the design assumption that hand-copying compact QR grids from small air-gapped displays can be operationally feasible when aided by templates and validation. The error rate, time cost, and fatigue profile for DuraShare-specific payloads remain to be measured.

Manual entropy generation. Manual generation of GF(2053) elements can use a $6\times7\times7\times7 = 2058$ mixed-radix rejection sampler. DuraDice-39 is the specialized physical implementation: four marked rods generate one outcome in $\{0, \ldots, 2057\}$, with visual rejection before summation. A low-cost DIY implementation uses four opaque bags and four matching token sets with pre-weighted values, for example poker chips, bottle caps, or cardboard disks marked with tape or ink. Under the physical assumption that each draw is uniform over tokens that are indistinguishable by touch and sufficiently uniform in size, weight, and texture within the same bag, the DIY method samples the same distribution as the specialized hardware. In either implementation, reject the five outcomes $2053, \ldots, 2057$; the accepted value is then uniform in GF(2053).

Usability context. An informal pilot with four non-technical participants (ages 28–72) on an early prototype suggested that (i) procedural retention was poor after 13 months, indicating that self-documenting materials and checkpointed workflows are important for inheritance scenarios, and (ii) manual Lagrange computation was the primary bottleneck, motivating the pre-computed coefficient tables in the current protocol. This pilot should be interpreted as hypothesis-generating only; it pre-dated the current checksum architecture and tested a harder configuration (4-of-16), so it is not representative of current workflow performance.

10 Discussion

The current manuscript should be read as an exploratory research artifact rather than as an audited operational system.

10.1 Limitations

- **MAT is a bounded manual authenticator, not a high-bit MAC:** single and dual MAT provide at most approximately 11 and 22 bits of false-accept resistance for one fixed artifact under secret-key custody. They complement rather than replace the computational Audit Hash, RBT, RVA, physical controls, and threshold redundancy.
- **The linear consistency layer is error-detection and diagnostic correction, not authentication:** the checksum architecture is a q-ary SPC product code with $d = 4$, so it detects any passive error pattern affecting up to three cells and uniquely corrects one wrong cell under a single-error assumption. It cannot prevent intentional substitution by an adversary with physical access, and forced auto-correction is unsafe when the number of erroneous cells is unknown.
- **No professional audit:** prototype implementations have not undergone independent security audit or formal verification.
- **Usability evidence is preliminary:** manual timing observations are from the protocol author; no controlled third-party usability study exists for v0.7.0. Current prototypes lag the complete workflow, so end-to-end software-assisted timing is not reported. The Full/Compact QR hand-mark comparison (Section 7.1, Appendix C.1) is a structural module count, not a timed transcription experiment.
- **Manual fallback without MAT remains unauthenticated:** SPC/GIC detects passive inconsistency but not deliberate document replacement. Without MAT, manual pre-recovery assurance relies on physical controls; without RVA, post-recovery wallet-context assurance is also reduced.
- **Manifest compromise affects verification:** compromise of a Whole-Key MAT section enables forgery of the matching Shares, while alteration of both a Share Payload and its Audit Hash record defeats the computational commitment. One Split-Key half reveals no complete MAT key but can be withheld or corrupted to cause denial. Recovery itself remains independent of MAT and the Manifest.

10.2 Falsifiability Criteria

F1 (Confidentiality): Formal analysis demonstrates coupled constraints enable $t < k$ shares to narrow search below 2^{256}.

F1b (Structured metadata leakage): A constructive adversary recovers from $t < k$ full arithmetic shares and protocol metadata more information about the protected mnemonic than the manifest and computational-tag analyses in Sections 3.2 and 8.3 allow, while remaining inside this paper's threat model (Section 3.3).

F2 (Manual feasibility): Controlled study ($n \geq 20$) shows $< 50\%$ success rate for manual recovery.

F3 (BIP39 compatibility): Recovered mnemonics fail validation or produce incorrect addresses in major implementations.

F4 (Product-code distance and correction): A 3-cell error pattern is found that passes all $r + 4$ check equations, or a one-cell error pattern is found whose syndrome does not uniquely identify the erroneous cell and value.

F5 (MAT authentication): Under the assumptions of Theorem 6.2.1, a fixed nonzero word-row substitution is accepted with probability greater than $1/2053$ per independently keyed required MAT column, or the passive Share/tag transcript distinguishes candidate Share Weights.

F6 (Implementation): Security audit identifies a vulnerability with attack complexity $< 2^{128}$.

10.3 Future Work

1. External cryptographic review: seek cryptographic peer review and independent reproduction of the reference test vectors.

2. Controlled usability evidence: gather third-party usability data for Sharing, Share Audit, and Recovery, including MAT generation/verification time, transcription errors, false alarms, inheritance-like conditions, and the operational difference between single and dual MAT. Also validate whether the $\approx$55% structural reduction in QR hand-mark burden (Appendix C.1) translates into proportionate savings in ceremony time and error rate for Compact.

3. Implementation assurance: pursue independent code review and professional security audit as appropriate to the intended deployment context.

4. Workflow design and sociotechnical analysis: the protocol defines the boundaries of Sharing, Share Audit, and Recovery, but deployment-specific choices remain: Share distribution, custodian selection, facilitation roles, rehearsal, rotation, and inheritance governance. Future work should develop opinionated deployment profiles with explicit threat narratives and controlled ceremony exercises.

The present manuscript should therefore be read as an analysis of a promising design point rather than as a claim of deployment readiness.

11 Conclusion

DuraShare targets a specific point in the self-custody design space: a BIP39-native threshold-backup format for cold storage and disaster recovery. Its arithmetic layer instantiates standard Shamir sharing directly over 1-indexed BIP39 word indices in GF(2053), preserving standard BIP39 input and output without custom share alphabets, seed transforms, or permanent dependence on a particular software stack.

The recommended operational path is human-first and software-assisted: software performs arithmetic, validates intermediate state, guides artifact handling, and supports recovery when available and trusted. The continuity property is that recovery remains implementation-independent. A valid threshold set can recover the mnemonic from durable artifacts and modular arithmetic alone. Optional MAT adds a calculator-verifiable, pre-recovery substitution check without changing that recovery path.

The linear consistency layer is an affine q-ary SPC product-code layer. It provides deterministic detection of all 1-, 2-, and 3-cell passive errors and exact single-cell correction under an explicit single-error assumption, but it is not an authentication mechanism. Substitution is addressed through layered detection and operational controls: MAT, Transport Hash, Manifest Audit Hash, protocol-input-bound RBT, wallet-bound RVA, artifact separation, tamper-evident handling, and custody procedures. MAT remains information-theoretic under its secret-key assumptions; the digital mechanisms remain computational and outside the arithmetic Share claim.

Information-theoretic confidentiality for $t < k$ applies to the arithmetic Share layer, and complete-Share confidentiality is unchanged by MAT. Compact is an optional constrained-output profile that reduces digital-envelope metadata in exchange for a smaller QR and weaker digital warning layers. Recursive composition is supported for advanced custody topologies, with independent MAT keys per active layer when selected.

This manuscript is offered as an applied-cryptography design for review, replication, and falsification. The test vectors support independent checking, and prototype implementations are in progress, but the system should not be treated as deployment-ready without further cryptographic review, implementation audit, and usability evidence.

Acknowledgments

The author acknowledges Adi Shamir's foundational 1979 secret sharing scheme [22], the BIP39 standard [4], and analysis of existing schemes (SLIP39, SSKR, Codex32, SeedXOR) that informed this design. The author thanks Jean Martina (LabSEC, INE–UFSC), whose review of an earlier version identified active recovery sabotage and share substitution as the protocol's principal operational risks. His initial observation that the protocol lacked a manual, pre-recovery way to validate shares directly motivated the design of the Manual Authentication Layer (MAT, Section 6) for manual deployments and the Manifest Audit Hash (Section 7.3) for computational deployments. His feedback also directly prompted the Recovery Verification Address design (Section 7.2), the explicit "zero substitution detection" statement for the linear consistency layer, and the separation of pre-recovery and post-recovery detection mechanisms (both in Section 8.2) in the current version. The author thanks Julio López (LASCA, IC–UNICAMP) for an informal remark that repeated Shamir sharing over a small prime field merited explicit leakage analysis; that comment prompted the local-leakage-resilience discussion in Section 8.3 and helped clarify the current Output Profile design. The author thanks Jeroen van de Graaf (DCC–ICEx–UFMG) for discussions on long-term key backup and manual feasibility, which prompted clearer presentation of the preliminary manual-fallback timing observations and the role of checkpointed workflows; and for suggesting that a user may intentionally pre-encrypt a mnemonic before entering it into DuraShare, which motivated the Pre-Encrypted Numeric Input mode for externally encrypted field-element payloads. The author thanks Edil Medeiros (ENE–UnB; Vinteum/Casa21) for clarifying the role of BIP39 wordlist language in wallet derivation, which prompted the BIP39 language component of the Language/Input byte and its incorporation into the Recovery Binding Tag canonical material. The author thanks jaonoctus (ZBD; Vinteum/Casa21) for instigating the pursuit of BCH codes' error-correction properties and possible protocol benefits. That prompt led to a study of BCH and Reed–Solomon constructions, to rejecting those families for the active protocol on complexity-to-benefit grounds, and to recognizing that the existing $d = 4$ SPC product-code layer already has a correction property: it detects any 1-, 2-, or 3-cell passive error and uniquely corrects a single-cell error under an explicit single-error assumption, now stated as Theorem 5.2.5. This work was developed independently without external funding. All errors remain the author's responsibility.

References

[1] A. Beimel, "Secret-Sharing Schemes: A Survey," *Coding and Cryptology, IWCC 2011*, Springer LNCS, vol. 6639, pp. 11–46, 2011.

[2] F. Benhamouda, A. Degwekar, Y. Ishai, and T. Rabin, "On the Local Leakage Resilience of Linear Secret Sharing Schemes," *CRYPTO 2018*, Springer LNCS, vol. 10991, pp. 531–561, 2018. Available: https://eprint.iacr.org/2019/653.

[3] P. Wuille, "BIP-32: Hierarchical Deterministic Wallets," *Bitcoin Improvement Proposals*, 2012. [Online]. Available: https://bips.dev/32/.

[4] M. Palatinus, P. Rusnak, A. Voisine, S. Bowe, "BIP-39: Mnemonic code for generating deterministic keys," *Bitcoin Improvement Proposals*, 2013. [Online]. Available: https://bips.dev/39/.

[5] P. Wuille and G. Maxwell, "BIP-173: Base32 address format for native witness outputs," *Bitcoin Improvement Proposals*, 2017. [Online]. Available: https://bips.dev/173/.

[6] P. Wuille, "BIP-350: Bech32m format for v1+ witness addresses," *Bitcoin Improvement Proposals*, 2020. [Online]. Available: https://bips.dev/350/.

[7] G. R. Blundo, A. De Santis, D. R. Stinson, and U. Vaccaro, "Graph decompositions and secret sharing schemes," *EUROCRYPT '92*, Springer LNCS, vol. 658, pp. 1–24, 1993.

[8] V. Buterin, "Why we need wide adoption of social recovery wallets," Blog post, Jan. 2021. https://vitalik.eth.link/general/2021/01/11/recovery.html.

[9] A. Poelstra, L. O'Connor, S. Corallo, "BIP-93: codex32: Checksummed SSSS-aware BIP32 seeds," *Bitcoin Improvement Proposals*, Draft, 2023. [Online]. Available: https://bips.dev/93/.

[10] R. Cramer, I. Damgård, and U. Maurer, "General secure multi-party computation from any linear secret-sharing scheme," *EUROCRYPT 2000*, Springer LNCS, vol. 1807, pp. 316–334, 2000.

[11] S. Eskandari, D. Barrera, E. Stobert, and J. Clark, "A First Look at the Usability of Bitcoin Key Management," *Proc. USEC 2015 (Workshop on Usable Security)*, 2015. doi: https://doi.org/10.14722/usec.2015.23015.

[12] P. Elias, "Error-free coding," *IRE Transactions on Information Theory*, vol. PGIT-4, no. 4, pp. 29–37, Sep. 1954.

[13] National Institute of Standards and Technology (NIST), *FIPS PUB 180-4: Secure Hash Standard (SHS)*, Aug. 2015. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf.

[14] L. K. Grover, "A fast quantum mechanical algorithm for database search," *Proc. 28th ACM Symposium on Theory of Computing (STOC '96)*, pp. 212–219, 1996.

[15] V. Guruswami and M. Wootters, "Repairing Reed-Solomon Codes," *Proc. 48th ACM Symposium on Theory of Computing (STOC '16)*, pp. 216–226, 2016.

[16] O. Klein and I. Komargodski, "New Bounds on the Local Leakage Resilience of Shamir's Secret Sharing Scheme," *CRYPTO 2023*, Springer LNCS, 2023. Available: https://eprint.iacr.org/2023/805.

[17] H. K. Maji, H. H. Nguyen, A. Paskin-Cherniavsky, T. Suad, and M. Wang, "Leakage-Resilience of the Shamir Secret-Sharing Scheme against Physical-Bit Leakages," *EUROCRYPT 2021*, Springer LNCS, 2021. Available: https://eprint.iacr.org/2021/186.

[18] H. K. Maji, H. H. Nguyen, A. Paskin-Cherniavsky, and M. Wang, "Leakage-Resilience of Shamir's Secret Sharing: Identifying Secure Evaluation Places," *Information-Theoretic Cryptography (ITC) 2025*, LIPIcs. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITC.2025.3.

[19] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, Feb. 1997. https://www.rfc-editor.org/rfc/rfc2104.

[20] R. Schiavinato Lopez, *Schiavinato Sharing v0.4.1* (historical name; now DuraShare), GRIFORTIS, 2025. Available at https://github.com/GRIFORTIS/durashare.

[21] Coinkite, "SeedXOR: XOR-based secret sharing for BIP39 mnemonics," 2021. [Online]. Available: https://seedxor.com/.

[22] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, Nov. 1979.

[23] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997 (conference version: *Proc. 35th IEEE FOCS*, 1994).

[24] P. Rusnak, A. Kozlik, O. Vejpustek, T. Susanka, M. Palatinus, and J. Hoenicke, "SLIP-0039: Shamir's Secret-Sharing for Mnemonic Codes," SatoshiLabs Improvement Proposals, Final, 2019. [Online]. Available: https://github.com/satoshilabs/slips/blob/master/slip-0039.md.

[25] W. McNally and C. Allen, "UR Type Definition for Sharded Secret Key Reconstruction (SSKR)," Blockchain Commons Research, BCR-2020-011, ver. 1.0.1, 2020 (rev. 2021). [Online]. Available: https://github.com/BlockchainCommons/Research/blob/master/papers/bcr-2020-011-sskr.md.

[26] D. R. Stinson, *Cryptography: Theory and Practice*, 3rd ed. Chapman and Hall/CRC, 2006.

[27] R. M. Tanner, "A recursive approach to low complexity codes," *IEEE Transactions on Information Theory*, vol. IT-27, no. 5, pp. 533–547, Sep. 1981.

[28] M. Tompa and H. Woll, "How to share a secret with cheaters," *Journal of Cryptology*, vol. 1, no. 2, pp. 133–138, 1988.

[29] M. N. Wegman and J. L. Carter, "New hash functions and their use in authentication and set equality," *Journal of Computer and System Sciences*, vol. 22, no. 3, pp. 265–279, 1981. doi: https://doi.org/10.1016/0022-0000(81)90033-7.

A Pre-Computed Lagrange Coefficients

Lagrange coefficients are determined entirely by share indices and contain zero secret information. They may be computed on any device (even untrusted) or verified against multiple sources.

Table 5: Lagrange coefficients in GF(2053)

Scheme	**Shares**	**Coefficients** (γ)
2-of-3	$\{1,2\}$	$(2, 2052)$
	$\{1,3\}$	$(1028, 1026)$
	$\{2,3\}$	$(3, 2051)$
2-of-4	$\{1,2\}$	$(2, 2052)$
	$\{1,3\}$	$(1028, 1026)$
	$\{1,4\}$	$(1370, 684)$
	$\{2,3\}$	$(3, 2051)$
	$\{2,4\}$	$(2, 2052)$
	$\{3,4\}$	$(4, 2050)$
3-of-5	$\{1,2,3\}$	$(3, 2050, 1)$
	$\{1,2,4\}$	$(687, 2051, 1369)$
	$\{1,2,5\}$	$(1029, 1367, 1711)$
	$\{1,3,4\}$	$(2, 2051, 1)$
	$\{1,3,5\}$	$(1285, 512, 257)$
	$\{1,4,5\}$	$(686, 1367, 1)$
	$\{2,3,4\}$	$(6, 2045, 3)$
	$\{2,3,5\}$	$(5, 2048, 1)$
	$\{2,4,5\}$	$(1372, 2048, 687)$
	$\{3,4,5\}$	$(10, 2038, 6)$

Computing coefficients for other share sets. For any recovery set $S = \{x_1, \ldots, x_k\}$ of distinct nonzero share indices, the interpolation coefficient for share x_j at the origin is

$$\gamma_j = \prod_{\substack{i=1 \\ i \neq j}}^{k} \frac{x_i}{x_i - x_j} \pmod{2053}.$$

All divisions are field divisions in GF(2053), implemented by multiplying with modular inverses. The recovered secret is then

$$f(0) = \sum_{j=1}^{k} \gamma_j \, y_j \pmod{2053},$$

where y_j is the share value at index x_j. Because the coefficients depend only on the public share indices, they may be precomputed once for any valid threshold set and reused across all rows, checksums, and GIC interpolation steps.

B Worked Example in GF(11)

A self-contained audit instance of the arithmetic layer, with all arithmetic modulo 11 instead of modulo 2053. The toy wordlist uses labels $1, \ldots, 10$; field element 0 may appear as a share or check value. Six words are arranged as two rows and three columns:

$$\begin{array}{lll} w_1 = 1 & w_2 = 10 & w_3 = 3 \\ w_4 = 5 & w_5 = 6 & w_6 = 1 \end{array}$$

Setup. Scheme: 2-of-3, share indices $x \in \{1, 2, 3\}$. Each polynomial is linear ($k = 2$): $f_{w_i}(x) = w_i + a_i x \pmod{11}$.

i	1	2	3	4	5	6
w_i	1	10	3	5	6	1
a_i	4	1	0	4	10	3

Row tags: $\tau_1^R = 1$, $\tau_2^R = 2$, so $T_R = 3$. Column tags: $\tau_1^C = 10$, $\tau_2^C = 20$, $\tau_3^C = 30$, so $T_C = 60$. (These toy magnitudes are chosen independently of the production tags in Section 4.2: modulo the toy field's small 11 elements, $\{10, 20, 30\}$ already reduces to $\{10, 9, 8\}$, the maximum separation available from the row tags $\{1, 2\}$; the production values $\{100, 200, 300\}$ would reduce to $\{1, 2, 3\}$ (mod 11), colliding with the row tags, so they are not reused here.)

B.1 Sharing

Step 1: Word polynomials.

$$f_{w_1}(x) = 1 + 4x \pmod{11} \quad f_{w_2}(x) = 10 + x \pmod{11} \quad f_{w_3}(x) = 3 \pmod{11}$$
$$f_{w_4}(x) = 5 + 4x \pmod{11} \quad f_{w_5}(x) = 6 + 10x \pmod{11} \quad f_{w_6}(x) = 1 + 3x \pmod{11}$$

Evaluate at $x \in \{1, 2, 3\}$ to produce one share grid per share index, with words placed in their row/column positions:

Share 1 ($x = 1$)

	1	2	3
1	5	0	3
2	9	5	4

Share 2 ($x = 2$)

	1	2	3
1	9	1	3
2	2	4	7

Share 3 ($x = 3$)

	1	2	3
1	2	2	3
2	6	3	10

Step 2: Row checksums and incremental validation.

$$f_{R_1}(x) = f_{w_1}(x) + f_{w_2}(x) + f_{w_3}(x) + 1 = 4 + 5x \pmod{11},$$
$$f_{R_2}(x) = f_{w_4}(x) + f_{w_5}(x) + f_{w_6}(x) + 2 = 3 + 6x \pmod{11}.$$

Each row checksum appends as a right column to its row.

Share 1 ($x = 1$)

	1	2	3	R
1	5	0	3	9
2	9	5	4	9

Share 2 ($x = 2$)

	1	2	3	R
1	9	1	3	3
2	2	4	7	4

Share 3 ($x = 3$)

	1	2	3	R
1	2	2	3	8
2	6	3	10	10

Validate $(w_{3j-2} + w_{3j-1} + w_{3j} + j) \bmod 11 = R_j$ for each row j and share x:

x	Row 1	Row 2
1	(5+0+3+1) mod 11 = 9 = R_1 ✓	(9+5+4+2) mod 11 = 9 = R_2 ✓
2	(9+1+3+1) mod 11 = 3 = R_1 ✓	(2+4+7+2) mod 11 = 4 = R_2 ✓
3	(2+2+3+1) mod 11 = 8 = R_1 ✓	(6+3+10+2) mod 11 = 10 = R_2 ✓

Privacy note. For row 1, the checksum polynomial can also be written as $(1 + 10 + 3 + 1) + (4 + 1 + 0)x = 4 + 5x$. This does not reveal the coefficient sum 5 to a single share holder: at $x = 1$, the observed check is simply $9 = (5 + 0 + 3 + 1) \bmod 11$, a value recomputable from that same visible row. The hidden decomposition into secret terms and coefficient terms becomes available only with a threshold set, exactly as for the word polynomials.

Step 3: Column checksums.

$$f_{C_1}(x) = f_{w_1}(x) + f_{w_4}(x) + 10 = 5 + 8x \pmod{11},$$
$$f_{C_2}(x) = f_{w_2}(x) + f_{w_5}(x) + 20 = 3 \pmod{11},$$
$$f_{C_3}(x) = f_{w_3}(x) + f_{w_6}(x) + 30 = 1 + 3x \pmod{11}.$$

Each column checksum spans all data rows of its column and appears as a bottom row.

Share 1 ($x = 1$)

	1	2	3	R
1	5	0	3	9
2	9	5	4	9
C	2	3	4	

Share 2 ($x = 2$)

	1	2	3	R
1	9	1	3	3
2	2	4	7	4
C	10	3	7	

Share 3 ($x = 3$)

	1	2	3	R
1	2	2	3	8
2	6	3	10	10
C	7	3	10	

Validate $\left(\sum_{i \in \mathrm{col}\, c} w_i + \tau_c^C\right) \bmod 11 = C_c$ for each column c and share x:

x	C_1	C_2	C_3
1	(5+9+10) mod 11 = 2 ✓	(0+5+20) mod 11 = 3 ✓	(3+4+30) mod 11 = 4 ✓
2	(9+2+10) mod 11 = 10 ✓	(1+4+20) mod 11 = 3 ✓	(3+7+30) mod 11 = 7 ✓
3	(2+6+10) mod 11 = 7 ✓	(2+3+20) mod 11 = 3 ✓	(3+10+30) mod 11 = 10 ✓

Step 4: GIC. The base GIC polynomial is

$$f_{\mathrm{GIC}}(x) = \sum_{i=1}^{6} f_{w_i}(x) + T_R + T_C = 89 + 22x = 1 \pmod{11}.$$

The share-bound GIC adds the share index for positional binding:

$$\mathrm{GIC}(x) = f_{\mathrm{GIC}}(x) + x = 1 + x \pmod{11}.$$

The GIC fills the lower-right corner, completing each share table:

Share 1 ($x = 1$)

	1	2	3	R
1	5	0	3	9
2	9	5	4	9
C	2	3	4	2

Share 2 ($x = 2$)

	1	2	3	R
1	9	1	3	3
2	2	4	7	4
C	10	3	7	3

Share 3 ($x = 3$)

	1	2	3	R
1	2	2	3	8
2	6	3	10	10
C	7	3	10	4

Three equivalent validation paths for share 1, whose GIC is 2:

Words: (5+0+3+9+5+4+3+60+1) mod 11 = 2 ✓
Rows: (9+9+60+1) mod 11 = 2 ✓
Cols: (2+3+4+3+1) mod 11 = 2 ✓

Optional MAT. For a single-MAT toy instance, choose independent per-Share weights and Row Pads:

Share x	(u_1, u_2, u_3)	(b_1, b_2)	MAT_1	MAT_2
1	$(2, 4, 7)$	$(6, 8)$	4	8
2	$(5, 1, 9)$	$(3, 7)$	10	7
3	$(8, 6, 2)$	$(4, 10)$	5	8

For Share 1, row 1 is checked by

$$(6 + 2 \cdot 5 + 4 \cdot 0 + 7 \cdot 3) \bmod 11 = 4,$$

and row 2 by

$$(8 + 2 \cdot 9 + 4 \cdot 5 + 7 \cdot 4) \bmod 11 = 8.$$

The tags append as a fifth column on the two word rows; the footer cell under that column is marked as not applicable and carries no MAT tag. Dual MAT would repeat the construction with an independent second key set.

B.2 Recovery from shares 1 and 3

Lagrange coefficients depend only on the share indices, not on any secret value, and may be looked up or computed on any device. All three pairs for this 2-of-3 scheme are:

Shares used	Coefficients for interpolation at $x = 0$	Sanity check
$1, 2$	$\gamma_1 = 2,\ \gamma_2 = 10$	$2 \cdot 1 + 10 \cdot 2 \equiv 0 \pmod{11}$
$1, 3$	$\gamma_1 = 7,\ \gamma_3 = 5$	$7 \cdot 1 + 5 \cdot 3 \equiv 0 \pmod{11}$
$2, 3$	$\gamma_2 = 3,\ \gamma_3 = 9$	$3 \cdot 2 + 9 \cdot 3 \equiv 0 \pmod{11}$

Step 1: Per-share GIC validation. Share 1:

$$(5{+}0{+}3{+}9{+}5{+}4{+}3{+}60{+}1) \bmod 11 = 2 = \text{GIC} \checkmark.$$

Share 3:

$$(2{+}2{+}3{+}6{+}3{+}10{+}3{+}60{+}3) \bmod 11 = 4 = \text{GIC} \checkmark.$$

Step 2: Coefficient lookup. From the table above, $\gamma_1 = 7$ and $\gamma_3 = 5$.

Step 3: Row-by-row interpolation with incremental validation. Interpolate row 1 words and row checksum, then validate before proceeding:

$$\hat{w}_1 = (7 \cdot 5 + 5 \cdot 2) \bmod 11 = 1$$
$$\hat{w}_2 = (7 \cdot 0 + 5 \cdot 2) \bmod 11 = 10$$
$$\hat{w}_3 = (7 \cdot 3 + 5 \cdot 3) \bmod 11 = 3$$
$$\hat{R}_1 = (7 \cdot 9 + 5 \cdot 8) \bmod 11 = 4$$

Incremental check:

$$(\hat{w}_1 + \hat{w}_2 + \hat{w}_3 + 1) \bmod 11 = (1 + 10 + 3 + 1) \bmod 11 = 4 = \hat{R}_1 \checkmark.$$

Interpolate row 2 and validate:

$$\hat{w}_4 = (7 \cdot 9 + 5 \cdot 6) \bmod 11 = 5$$
$$\hat{w}_5 = (7 \cdot 5 + 5 \cdot 3) \bmod 11 = 6$$
$$\hat{w}_6 = (7 \cdot 4 + 5 \cdot 10) \bmod 11 = 1$$
$$\hat{R}_2 = (7 \cdot 9 + 5 \cdot 10) \bmod 11 = 3$$

$$(\hat{w}_4 + \hat{w}_5 + \hat{w}_6 + 2) \bmod 11 = (5 + 6 + 1 + 2) \bmod 11 = 3 = \hat{R}_2 \checkmark.$$

Step 4: Global validation. Interpolate column checksums and GIC:

$$\hat{C}_1 = (7{\cdot}2{+}5{\cdot}7) \bmod 11 = 5, \quad \hat{C}_2 = (7{\cdot}3{+}5{\cdot}3) \bmod 11 = 3, \quad \hat{C}_3 = (7{\cdot}4{+}5{\cdot}10) \bmod 11 = 1.$$

Column checks:

$$(1{+}5{+}10) \bmod 11 = 5 = \hat{C}_1, \quad (10{+}6{+}20) \bmod 11 = 3 = \hat{C}_2, \quad (3{+}1{+}30) \bmod 11 = 1 = \hat{C}_3 \checkmark.$$

Interpolate the printed GIC values:

$$\hat{\text{GIC}} = (7 \cdot 2 + 5 \cdot 4) \bmod 11 = 1.$$

The $+x$ share-binding terms cancel by the Lagrange sanity check, leaving the base GIC at $x = 0$.

Three validation paths:

$$\text{Words: } (1{+}10{+}3{+}5{+}6{+}1{+}3{+}60) \bmod 11 = 1 = \hat{\text{GIC}} \checkmark$$
$$\text{Rows: } (4{+}3{+}60) \bmod 11 = 1 = \hat{\text{GIC}} \checkmark$$
$$\text{Cols: } (5{+}3{+}1{+}3) \bmod 11 = 1 = \hat{\text{GIC}} \checkmark$$

Step 5: Output. The recovered toy mnemonic is

$$(w_1, w_2, w_3, w_4, w_5, w_6) = (1, 10, 3, 5, 6, 1).$$

In the production protocol, the corresponding recovered values are BIP39 word indices in $1, \ldots, 2048$ and are then converted to BIP39 words and checked against the BIP39 checksum.

C Digital Envelope Profile Summary

The byte-level payload encoding is an implementation reference, maintained with the repository test vectors. This appendix records only the protocol-relevant profile differences. The arithmetic Share table remains the manual recovery artifact; the digital envelope is an assisted-deployment transport and verification layer.

Both active profiles carry protocol/profile metadata, Language/Input context, threshold and share-index context, a Session Batch ID, an RBT, and serialized share data. MAT selection, tags, complete keys, and Split-Key halves are human-readable artifact fields only; they are not serialized in either profile and do not change QR payload size.

Profile	**Serialized share data**	**Integrity and verification**	**24-word QR**
Full	Complete canonical arithmetic table: word shares, row checksums, column checksums, and printed GIC	Transport Hash, QR error correction, RBT, and arithmetic table validation	99 bytes; V6-M
Compact	Word shares only; row checksums, column checksums, and printed GIC remain paper fields	QR error correction, RBT, and recomputation against the printed paper table; no Transport Hash	53 bytes; V3-L

Full is the default recommendation for long-term, high-value storage because its payload commits to the complete canonical arithmetic table and includes a Transport Hash. Compact exists for constrained-output workflows where QR size or hand transcription dominates operational risk. Its smaller payload shifts more verification work back to the printed artifact: software must recompute row, column, and GIC values from scanned word shares and compare them against the human-readable table.

The representative 24-word payload sizes above are used in the QR hand-transcription workload estimate below. Exact field order, byte offsets, packing rules, and text-export round trips are implementation details and should be checked against the reference test vectors rather than inferred from this whitepaper.

C.1 QR Hand-Transcription Workload Estimate

This appendix records the methodology behind the hand-mark burden figures in Table 3 and Section 7.1. The goal is a reproducible, conservative *structural* comparison of Full versus Compact share-QR transcription; it is not a controlled usability study of time, error rate, or operator fatigue.

Scope. One *share* payload QR per comparison (not the Hand-Transcribed DRK resume QR, which is a separate resume artifact). Ceremonies are assumed to use a pre-printed blank template with ISO/IEC 18004 structural patterns already marked: three finder patterns (each including the inner 3×3 square), their separators, timing strips, the alignment pattern for the version, and the format-information strips for the chosen error-correction level. The operator copies only the payload-dependent region (data and error-correction codewords, including mask-dependent placement) from a trusted air-gapped display.

Counting rule. Let each symbol module be one cell in the $n \times n$ QR matrix, with $n = 17 + 4v$ for QR Version v ($n = 41$ for Version 6 and $n = 29$ for Version 3 in the 24-word comparison below). Modules reserved for the structural and format patterns above are excluded. The **hand-mark burden** is the number of remaining modules that are *dark* in the reference encoding. Light modules in that region must remain light on the template; a wrong dark mark is as fatal to decoding as a missing dark mark. The burden metric therefore models *marking effort* (count of dark cells to paint), not the full $2n_{\text{hand}}$ binary decisions; the latter overstates savings from white cells that stay blank on a pre-printed grid.

Reference encodings. The comparison uses the payload sizes in this appendix: Full 24-word share QR, 99 bytes, Version 6, error-correction level M; Compact 24-word share QR, 53 bytes, Version 3, error-correction level L. These match the baseline versions stated for each profile. Template module counts are reproduced analytically (finder+separator regions, timing strips, alignment pattern, format-information strips, dark module). The Full V6-M dark-module count is reported as a rounded structural estimate using the observed 49.4% dark ratio measured for V5-M as a same-EC-level proxy; the Compact V3-L count is the existing reference count. The repository reference script automates exact mask-dependent counts with a QR encoder.

Results (24-word representative payloads).

	Full (V6-M)	**Compact (V3-L)**
Symbol modules ($n \times n$)	1,681	841
Pre-printed structural + format modules	298	274
Hand region (all cells)	1,383	567
Hand-mark burden (dark modules)	**∼683**	**∼304**

Compact requires ≈55% fewer dark marks (304/683 ≈ 45% of the Full burden). Payload and mask variation typically shifts each count by ±20–40 modules; the rounded table entries (∼685, ∼305) are conservative midpoints for exposition.

Quiet zone. ISO decoders expect a four-module white margin around the symbol. That margin is excluded from all counts above; operators normally leave it blank rather than painting $(n + 8)^2 - n^2$ additional cells around the symbol.

Limitations. (i) No claim about ceremony duration or error probability. (ii) The dark-module count for V6-M is a structural estimate based on the observed dark ratio for V5-M; an exact count requires running the reference script with a QR encoder. (iii) 12- and 15-word Full mnemonics use V5-M (37×37, 84-byte capacity) rather than V6-M; their hand-mark counts are smaller than the 24-word representative, so the ≈55% Compact reduction should be read as the 24-word comparison, not a length-independent constant.

D Deployment and Artifact Trust Boundaries

This appendix summarizes the deployment model at the level of artifact classes and artifact-production tiers rather than application screens. Secret-bearing artifacts remain inside the trusted secret-handling boundary. Encrypted sensitive resume artifacts may move to a Companion device as ciphertext only. Manifests contain no plaintext Share values, but Whole-Key and Split-Key MAT fields are secret material; all Manifests remain separate from Shares.

D.1 Printer and Output Tiers

The software-assisted sharing workflow separates computation trust from output-device trust. Four output tiers are defined:

1. **Tier 1—offline non-network printer:** a USB-only or otherwise non-networked printer directly attached to the air-gapped ceremony device. Secret-bearing material may be printed. Full is recommended.
2. **Tier 2—personal network-capable printer:** a printer controlled by the user but capable of Wi-Fi, Ethernet, cloud print, stored jobs, or similar features. Secret-bearing material MUST NOT be printed. Non-secret Share and Manifest metadata, partially completed forms, and blank QR grids may be printed, accepting the privacy and targeting exposure; all secret fields remain blank.
3. **Tier 3—untrusted printer:** workplace, library, print-shop, shared, or otherwise uncontrolled printers. Only blank structure may be printed: grids, labels, checkboxes, empty fields, and optional blank QR overlays. No Session Batch ID, RBT, Printed GIC, Audit QR, or secret-bearing data should be sent to this tier.
4. **Tier 4—no printer:** all output is hand-transcribed from the air-gapped device display.

For Tiers 2–4, secret-bearing Share values, row checksums, column checksums, printed GIC, MAT tags, MAT key values or halves, and payload QR/text exports are copied by hand from the trusted device and then completely verified. Compact is recommended when QR transcription burden dominates, but Full remains available when the operator accepts the larger QR grid in exchange for the Transport Hash.

Artifact class	**Examples**	**Allowed handling**	**Lifecycle**
Plaintext secret-bearing	Mnemonic, plaintext Share tables including MAT tags, payload QR/text, plaintext coefficients, complete MAT keys, MAT key halves	Air-gapped ceremony device and Tier 1 trusted non-networked printer only; never sent to Companion or network/public printers	Long-term Shares and Manifests enter assigned custody; working copies are destroyed
Encrypted sensitive	Digital Resume payload, encrypted coefficient pool, encrypted DRK QR	May be stored by Companion as ciphertext only; Encryption Passphrase and decrypted contents remain on the air-gapped device	Temporary ceremony-continuity material; deleted after successful share creation
Operational metadata	Non-MAT Manifest fields, Batch ID, RBT, Audit Hashes, wallet-context hints, blank templates	May be printed on Tier 1 or Tier 2 according to printer-tier rules; Tier 3 receives only blank structure	Stored separately from shares for audit/inventory
Recovery inputs	k shares and optional manifest context	Recovered on paper or trusted air-gapped recovery device; manifest assists audit and context but is not required for arithmetic recovery	Used only during recovery or audit drill

Table 6: Deployment and artifact trust boundaries. Secret-bearing material remains within trusted secret-handling components. Encrypted sensitive resume artifacts may leave the air-gapped device only as ciphertext. Non-MAT Manifest metadata remains operationally sensitive; MAT key fields follow the secret-material rules.